Registre des opérations maraîchères

Rapport de projet — Brief 1

Application en ligne de commande développée en JavaScript avec Node.js, pour remplacer le carnet papier d'une exploitation maraîchère pédagogique.

Sommaire

Introduction générale 2

1. Initialisation de l'environnement de travail 2

2. Planification et gestion du projet 11

3. Conception et modélisation des données 12

4. Développement du module de stockage (src/stockage.js) 14

5. Développement du registre et des règles métier (src/registre.js) 19

6. Développement de la recherche et du tri (src/recherche.js) 22

7. Développement de la synthèse (src/synthese.js) 24

8. Développement du menu principal (app.js) 27

9. Développement du bonus (src/bonus.js et app.js) 30

10. Bilan final et tests de bout en bout 31

Conclusion 31

Annexe — Glossaire 32

Introduction générale

Contexte du projet

Ce projet s'inscrit dans le cadre de ma formation chez Digital Farming School (DFS). Il s'agit du « Brief 1 », un projet individuel qui consiste à développer une application en ligne de commande (terminal) en JavaScript avec Node.js.

Le besoin métier

La responsable d'une exploitation maraîchère pédagogique notait jusqu'à présent ses opérations culturales (semis, arrosage, désherbage, etc.) dans un carnet papier. Ce système provoquait des oublis de suivi et rendait la consultation des données difficile. L'objectif de ce projet est de remplacer ce carnet par un outil numérique permettant de saisir, rechercher, trier et totaliser les dépenses par parcelle.

Ce que fait l'application (et ce qu'elle ne fait pas)

L'outil permet d'enregistrer des opérations, de les sauvegarder dans un fichier JSON, de les consulter et de calculer des statistiques (totaux, minimums, maximums). En revanche, il ne calcule pas la rentabilité de l'exploitation, car le registre ne contient que les dépenses (les revenus ne sont pas saisis). C'est une limite importante que j'ai intégrée dans le code.

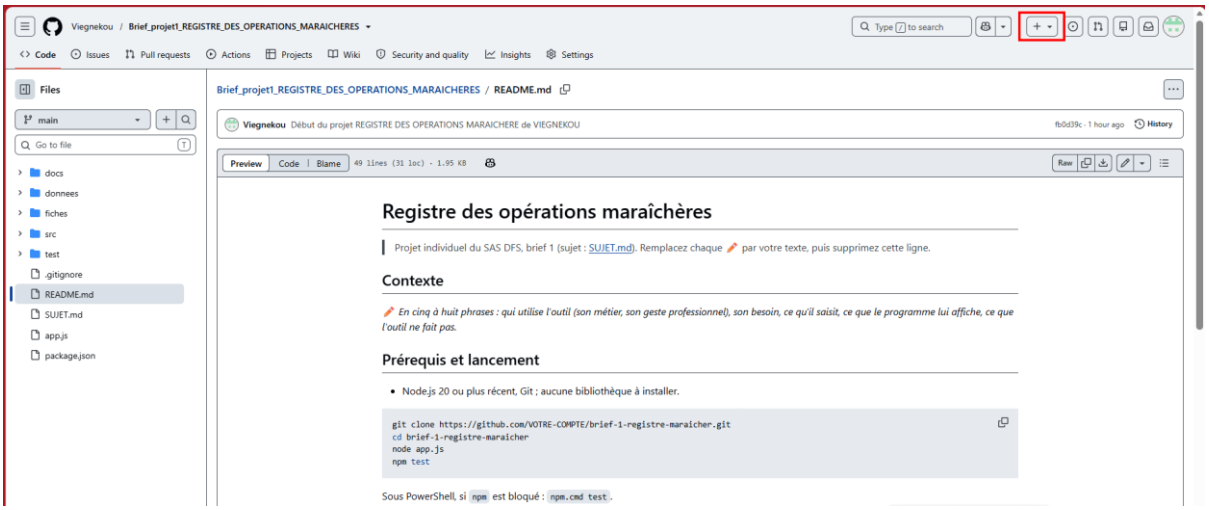
> EN CLAIR
>
> - **Opération culturale** : toute intervention réalisée sur une culture (semis, arrosage, désherbage…).
> - **Parcelle** : portion de terrain de l'exploitation ; elle sert à regrouper les dépenses.
> - **Pourquoi pas de rentabilité ?** : la rentabilité se calcule avec « revenus − dépenses ». Comme seules les dépenses sont saisies, le résultat serait trompeur : l'application s'en tient donc aux totaux de dépenses.

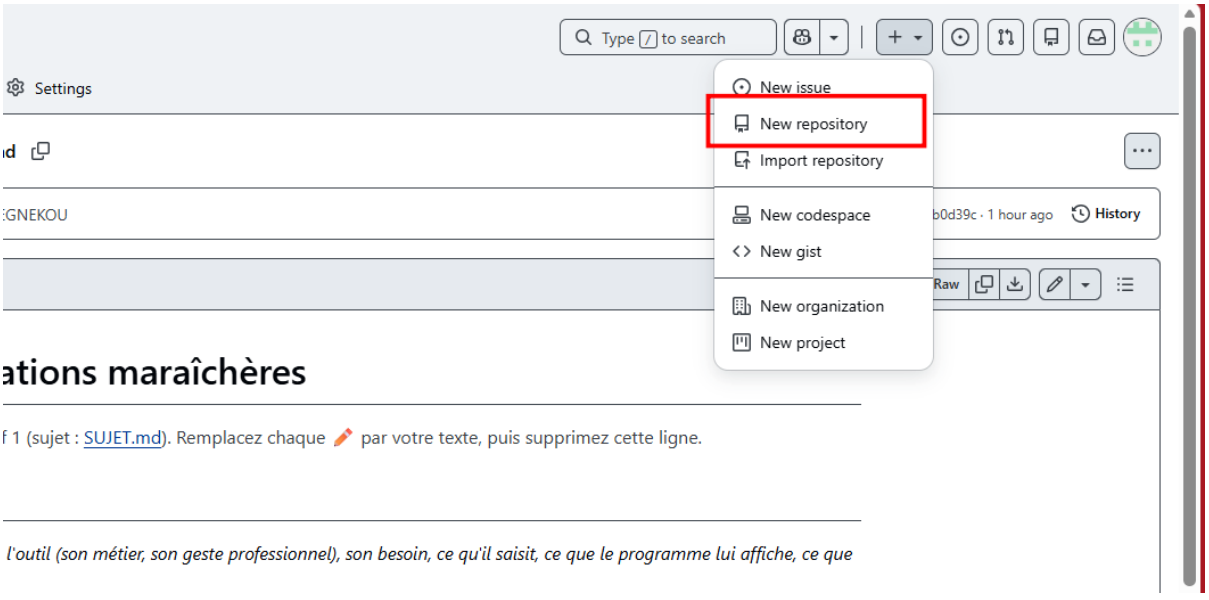
1. Initialisation de l'environnement de travail

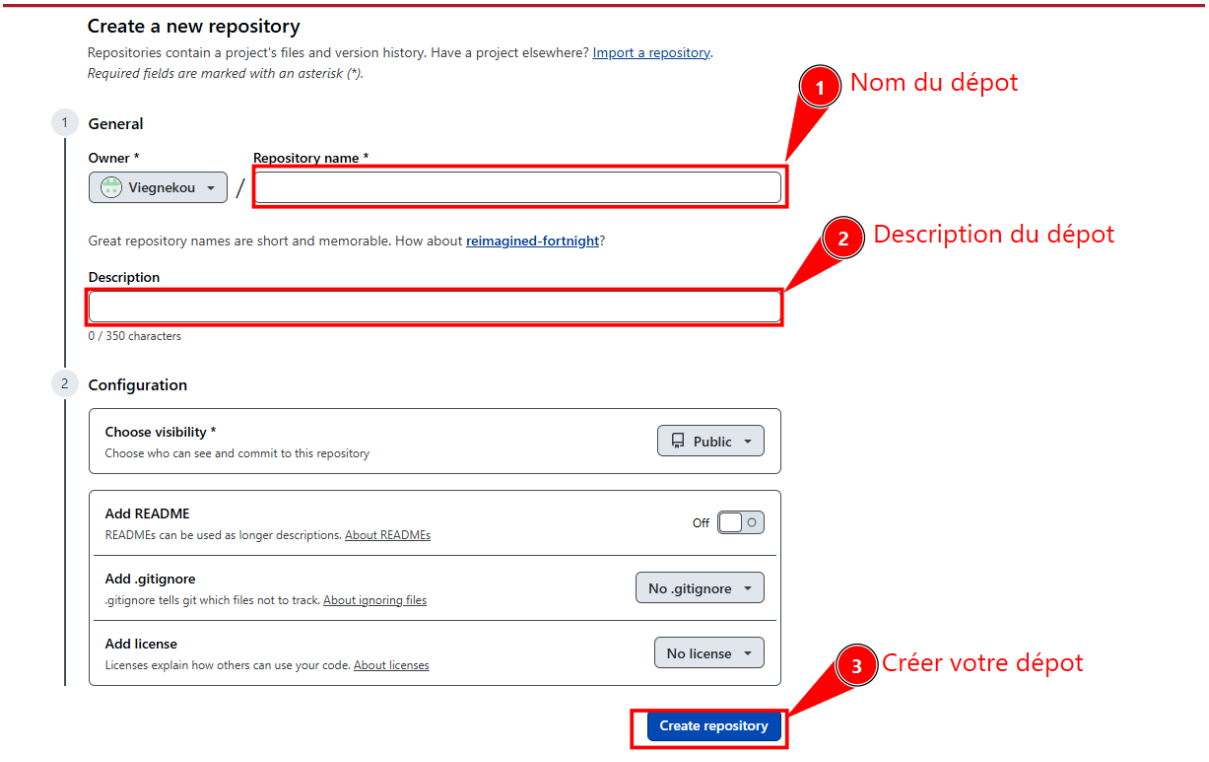
Avant de coder, j'ai préparé mon environnement. J'ai créé un dépôt vide sur mon compte GitHub pour héberger mon travail. Ensuite, j'ai utilisé le terminal de VS Code pour cloner le dépôt officiel de l'école (contenant le kit de départ). Enfin, j'ai relié mon dossier local à mon propre dépôt distant avec `git remote set-url origin`. Cela me permet de sauvegarder mon travail régulièrement et de le rendre accessible au coach.

EN CLAIR

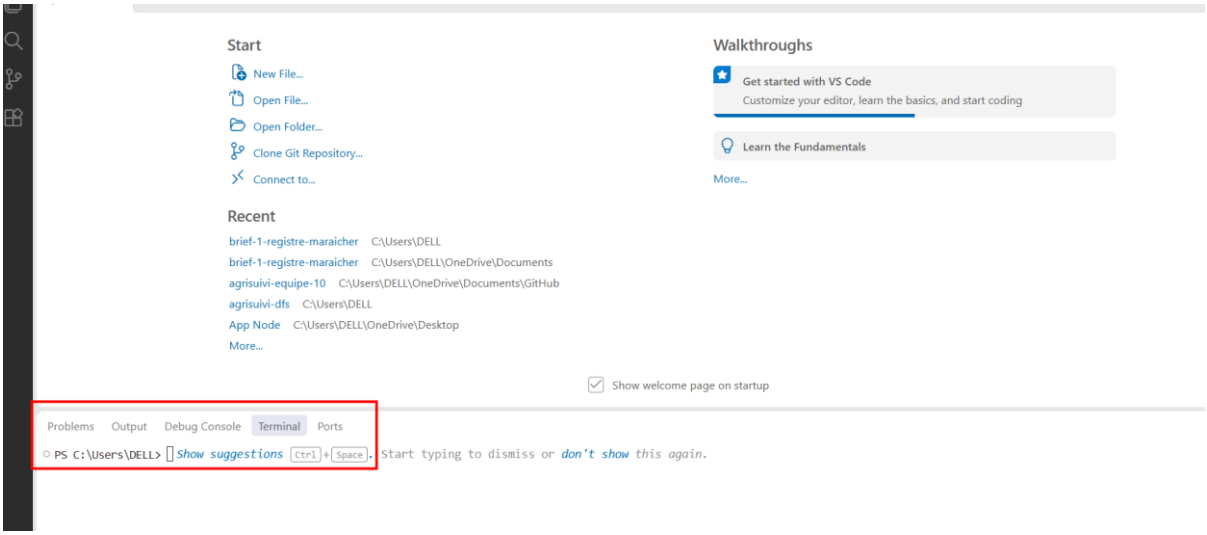
- **Cloner** : copier sur mon ordinateur le dépôt de l'école (le kit de départ).
- **`git remote set-url origin`** : modifie l'adresse du dépôt distant appelé `origin`. Après un clone, `origin` pointe vers le dépôt de l'école ; en le redirigeant vers mon dépôt GitHub, mes envois arrivent chez moi.
- **`git add .`** / **`commit`** / **`push`** : `add` prépare les fichiers modifiés, `commit` enregistre un point de sauvegarde accompagné d'un message, `push` envoie le tout sur GitHub.

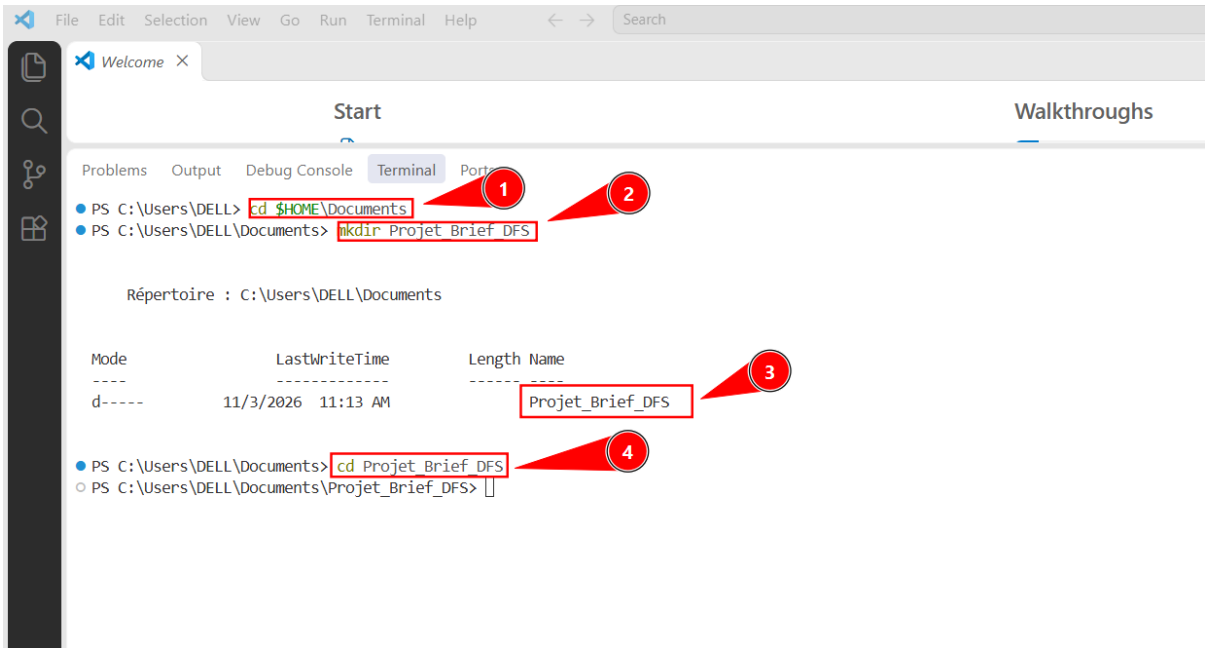

Registre des opérations maraîchères
Projet individuel du SAS DFS, brief 1 (sujet : SUJET.md). Remplacez chaque par votre texte, puis supprimez cette ligne.
Contexte
En cinq à huit phrases : qui utilise l'outil (son métier, son geste professionnel), son besoin, ce qu'il saisit, ce que le programme lui affiche, ce que l'outil ne fait pas.
Prérequis et lancement
Node.js 20 ou plus récent, Git ; aucune bibliothèque à installer.
git clone https://github.com/VOTRE-COMPTE/brief-1-registre-maraicher.git
cd brief-1-registre-maraicher
node app.js
npm test
Sous PowerShell, si npm est bloqué : npm.cmd test.

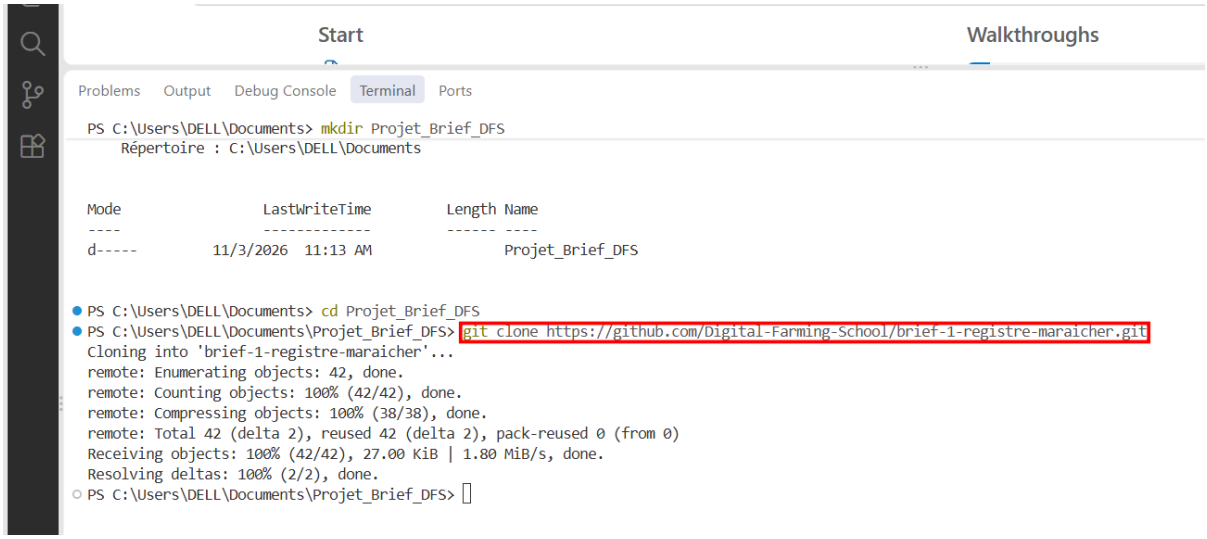

Type / to search
Settings
New issue
New repository
Import repository
New codespace
New gist
New organization
New project
History
Raw
ations maraîchères
f 1 (sujet : SUJET.md). Remplacez chaque par votre texte, puis supprimez cette ligne.
l'outil (son métier, son geste professionnel), son besoin, ce qu'il saisit, ce que le programme lui affiche, ce que

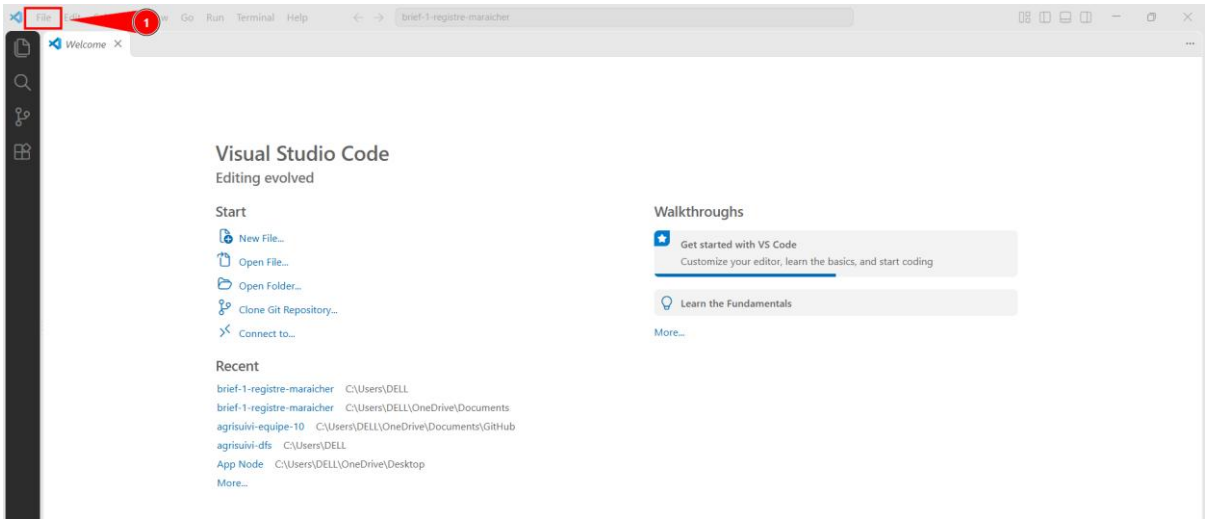

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).
1 Nom du dépot
1 General
Owner *
Repository name *
Viegnekou
Great repository names are short and memorable. How about reimagined-fortnight?
2 Description du dépot
Description
0 / 350 characters
2 Configuration
Choose visibility *
Choose who can see and commit to this repository
Public
Add README
READMEs can be used as longer descriptions. About READMEs
Off
Add .gitignore
.gitignore tells git which files not to track. About ignoring files
No .gitignore
Add license
Licenses explain how others can use your code. About licenses
No license
3 Créer votre dépot
Create repository

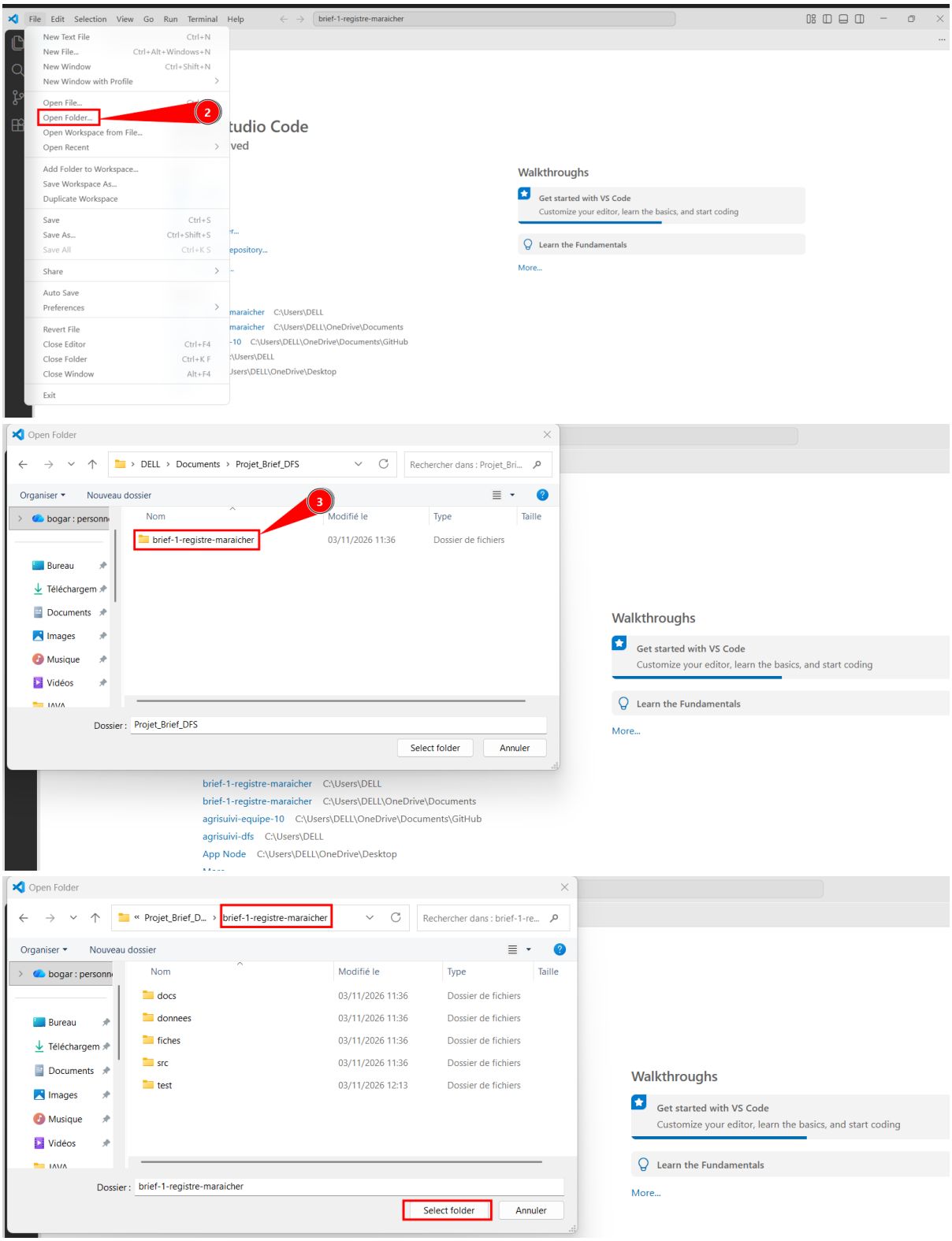
Capture 1 : *Page GitHub de création du nouveau dépôt*

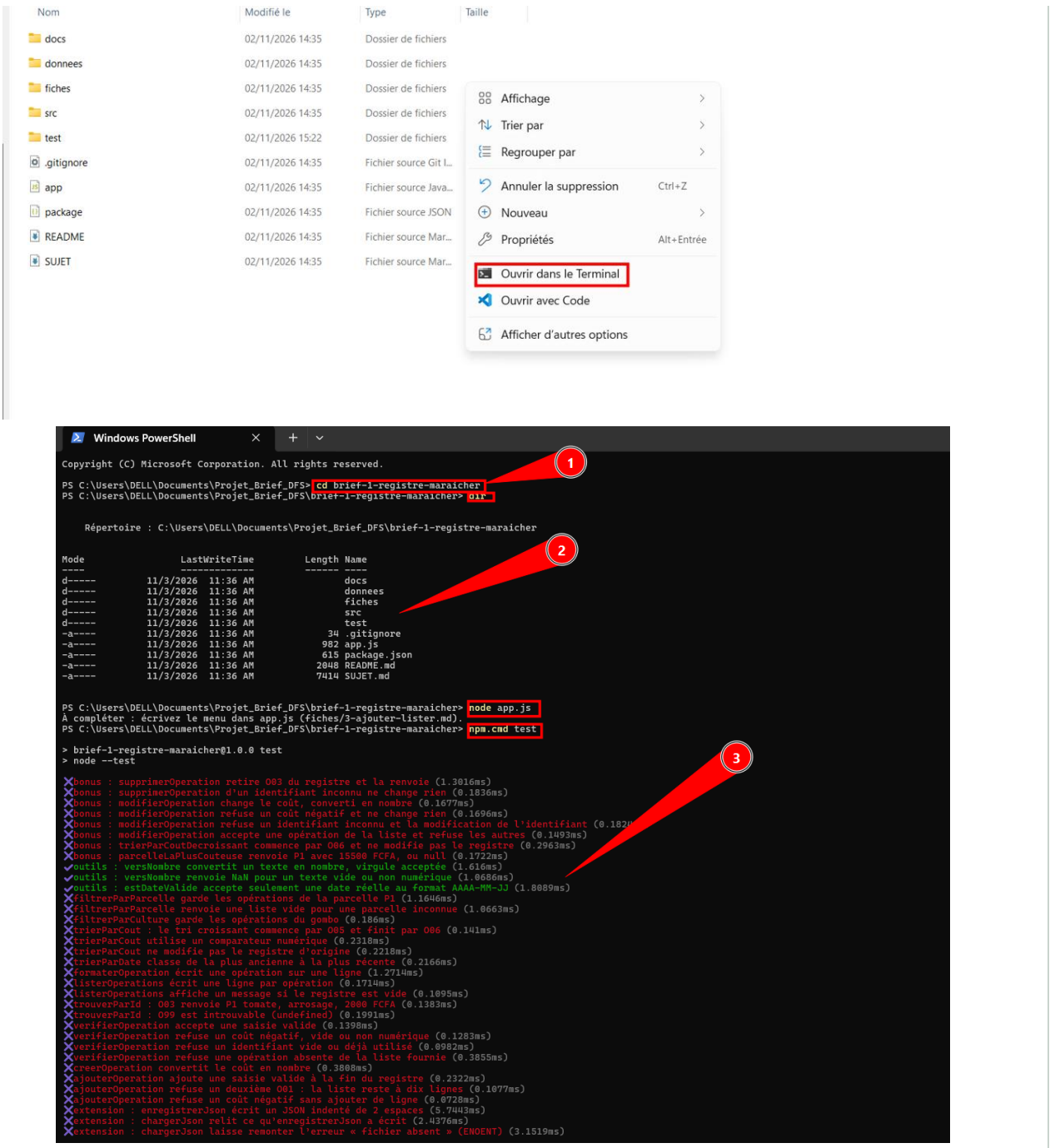



File Edit Selection View Go Run Terminal Help
Search
Welcome
Start
Walkthroughs
Problems Output Debug Console Terminal Ports
PS C:\Users\DELL> cd $HOME\Documents
PS C:\Users\DELL\Documents> mkdir Projet_Brief_DFS
Répertoire : C:\Users\DELL\Documents
Mode LastWriteTime Length Name
d----- 11/3/2026 11:13 AM Projet_Brief_DFS
PS C:\Users\DELL\Documents> cd Projet_Brief_DFS
PS C:\Users\DELL\Documents\Projet_Brief_DFS>
1
2
3
4

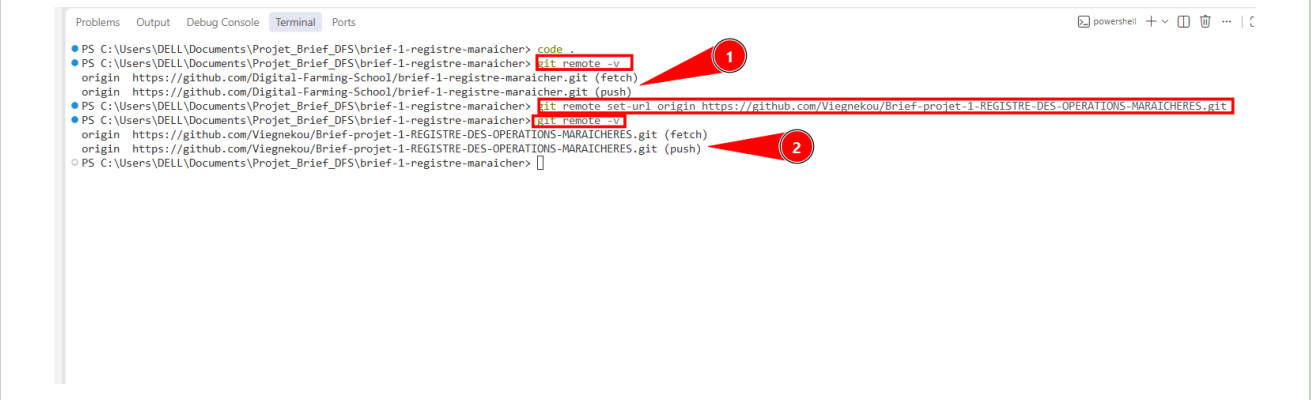

Start
Walkthroughs
Problems Output Debug Console Terminal Ports
PS C:\Users\DELL\Documents> mkdir Projet_Brief_DFS
Répertoire : C:\Users\DELL\Documents
Mode LastWriteTime Length Name
d----- 11/3/2026 11:13 AM Projet_Brief_DFS
PS C:\Users\DELL\Documents> cd Projet_Brief_DFS
PS C:\Users\DELL\Documents\Projet_Brief_DFS> git clone https://github.com/Digital-Farming-School/brief-1-registre-maraicher.git
Cloning into 'brief-1-registre-maraicher'...
remote: Enumerating objects: 42, done.
remote: Counting objects: 100% (42/42), done.
remote: Compressing objects: 100% (38/38), done.
remote: Total 42 (delta 2), reused 42 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (42/42), 27.00 KiB | 1.80 MiB/s, done.
Resolving deltas: 100% (2/2), done.
PS C:\Users\DELL\Documents\Projet_Brief_DFS>


File
1
Welcome
Visual Studio Code
Editing evolved
Start
New File...
Open File...
Open Folder...
Clone Git Repository...
Connect to...
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Learn the Fundamentals
More...
Recent
brief-1-registre-maraicher C:\Users\DELL
brief-1-registre-maraicher C:\Users\DELL\OneDrive\Documents
agrisuivi-equipe-10 C:\Users\DELL\OneDrive\Documents\GitHub
agrisuivi-dfs C:\Users\DELL
App Node C:\Users\DELL\OneDrive\Desktop
More...

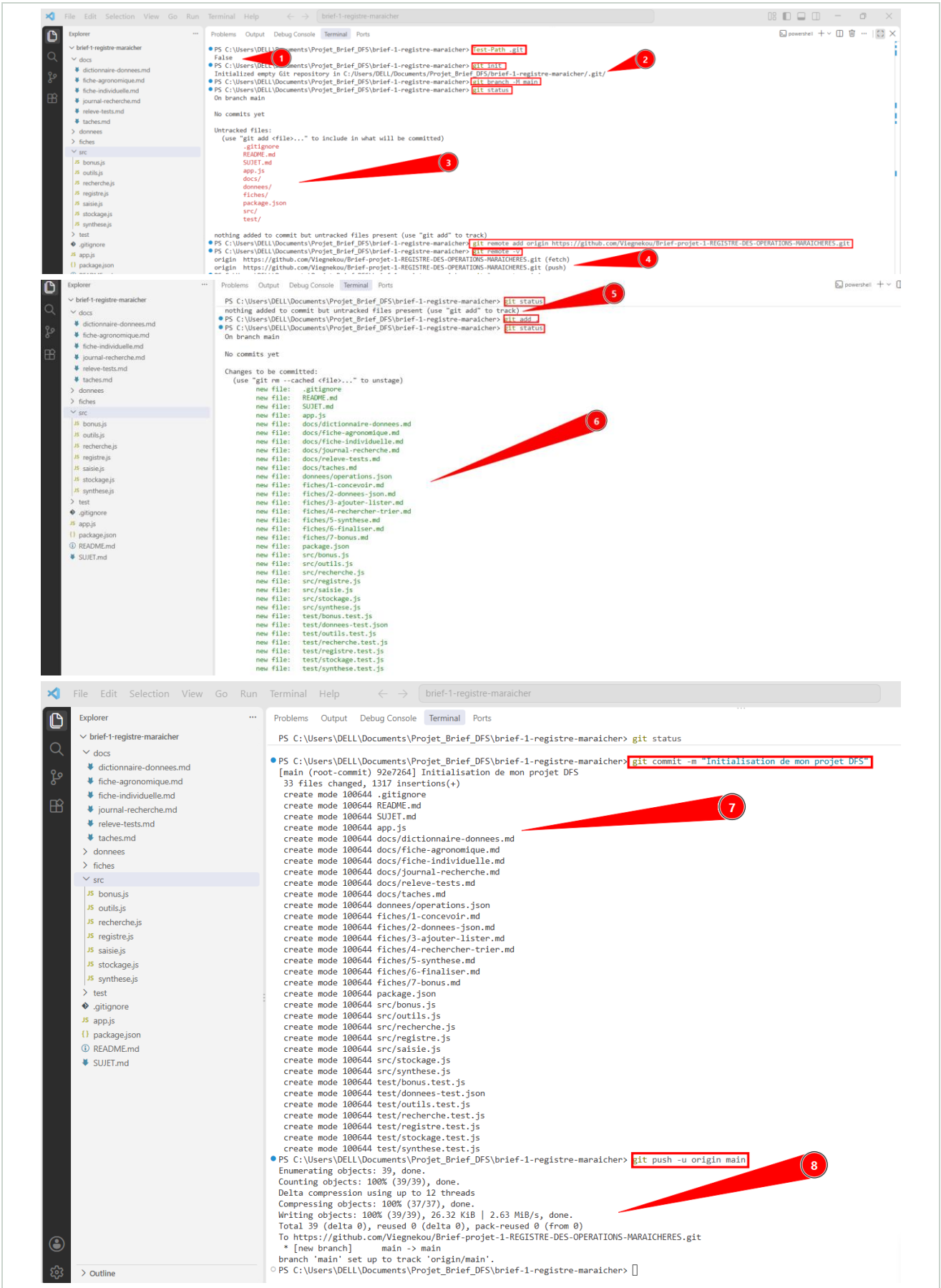

File Edit Selection View Go Run Terminal Help
brief-1-registre-maraicher
New Text File Ctrl+N
New File... Ctrl+Alt+Windows+N
New Window Ctrl+Shift+N
New Window with Profile
Open File...
Open Folder...
Open Workspace from File...
Open Recent
Add Folder to Workspace...
Save Workspace As...
Duplicate Workspace
Save Ctrl+S
Save As... Ctrl+Shift+S
Save All Ctrl+K S
Share
Auto Save
Preferences
Revert File
Close Editor Ctrl+F4
Close Folder Ctrl+K F
Close Window Alt+F4
Exit
2
tudio Code
ved
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Learn the Fundamentals
More...
maraicher C:\Users\DELL
maraicher C:\Users\DELL\OneDrive\Documents
-10 C:\Users\DELL\OneDrive\Documents\GitHub
Users\DELL\OneDrive\Desktop
Open Folder
DELL › Documents › Projet_Brief_DFS
Rechercher dans : Projet_Bri...
Organiser
Nouveau dossier
Nom
Modifié le
Type
Taille
brief-1-registre-maraicher
03/11/2026 11:36
Dossier de fichiers
3
Bureau
Téléchargem
Documents
Images
Musique
Vidéos
Dossier : Projet_Brief_DFS
Select folder
Annuler
brief-1-registre-maraicher C:\Users\DELL
brief-1-registre-maraicher C:\Users\DELL\OneDrive\Documents
agrisuivi-equipe-10 C:\Users\DELL\OneDrive\Documents\GitHub
agrisuivi-dfs C:\Users\DELL
App Node C:\Users\DELL\OneDrive\Desktop
Open Folder
« Projet_Brief_D... › brief-1-registre-maraicher
Rechercher dans : brief-1-re...
Organiser
Nouveau dossier
Nom
Modifié le
Type
Taille
docs 03/11/2026 11:36 Dossier de fichiers
donnees 03/11/2026 11:36 Dossier de fichiers
fiches 03/11/2026 11:36 Dossier de fichiers
src 03/11/2026 11:36 Dossier de fichiers
test 03/11/2026 12:13 Dossier de fichiers
Bureau
Téléchargem
Documents
Images
Musique
Vidéos
Dossier : brief-1-registre-maraicher
Select folder
Annuler
Walkthroughs
Get started with VS Code
Customize your editor, learn the basics, and start coding
Learn the Fundamentals
More...

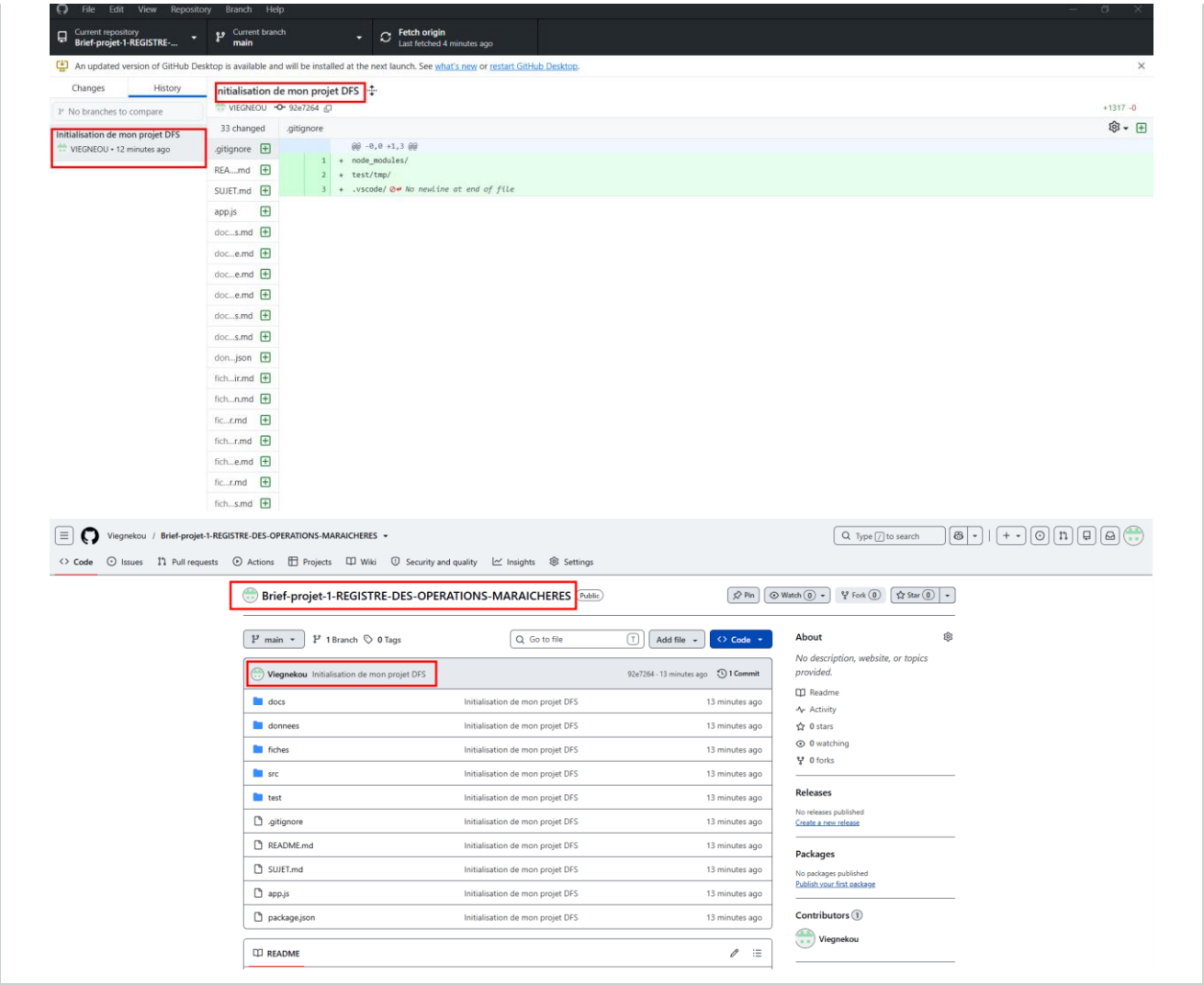


Capture 2 : *Terminal : lancement du projet sous cmd.*



Capture 3 : *Terminal VS Code avec git remote set-url origin.*


1
2
3
4
5
6
7
8

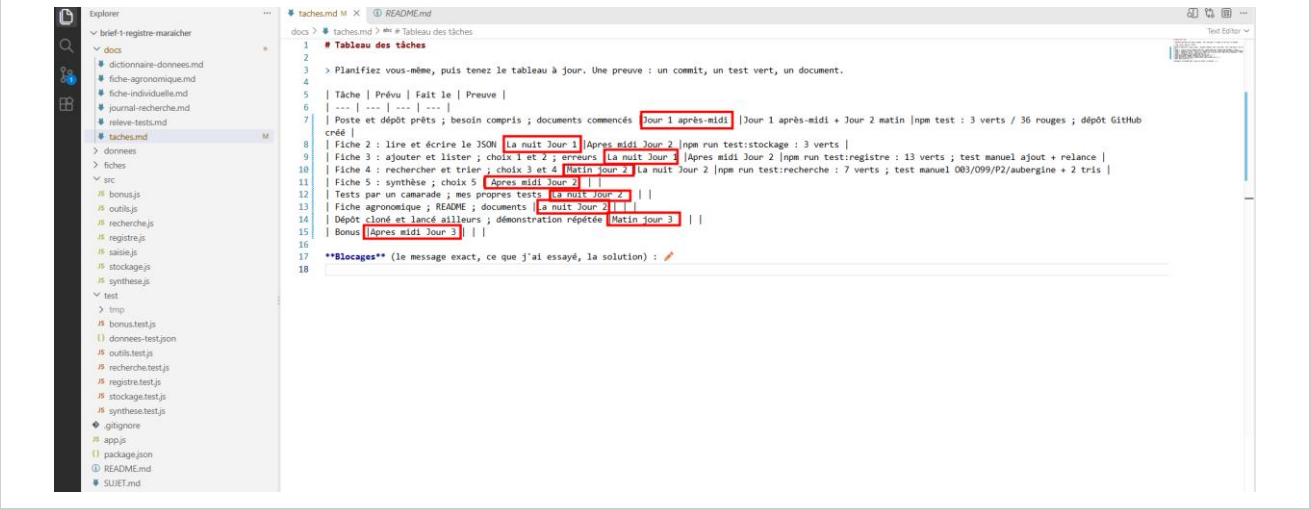


Capture 4 : *Premier commit et push (git add ., git commit, git push).*

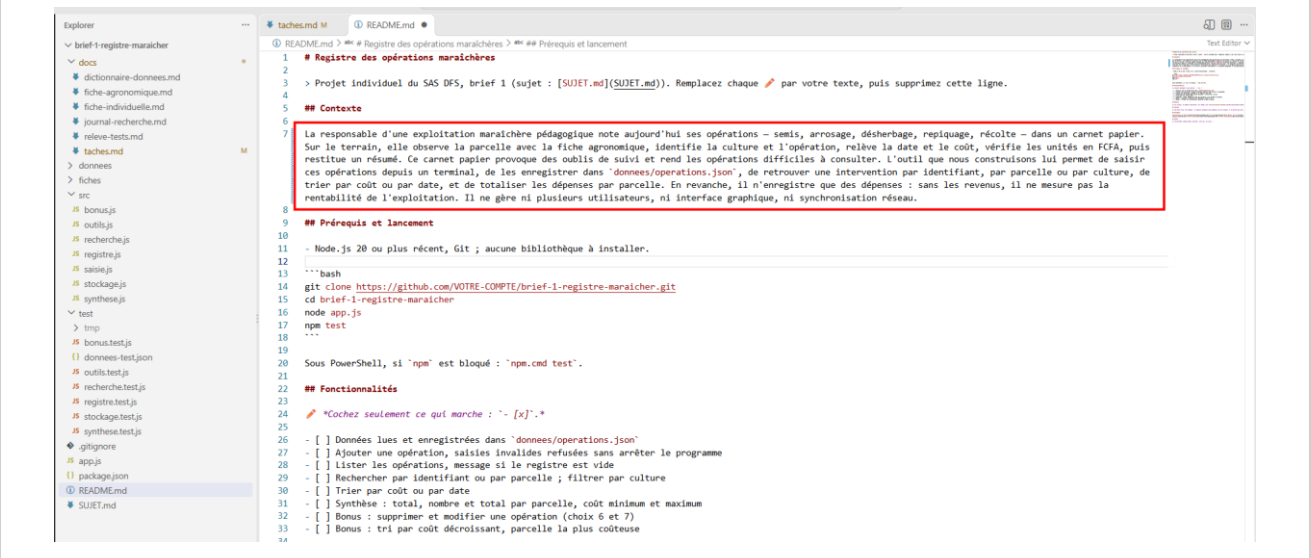
2. Planification et gestion du projet

Le projet s'étale sur 4 jours. Pour ne pas me perdre, j'ai rempli le fichier `docs/taches.md` en listant toutes les tâches, mon calendrier prévisionnel et les preuves de réussite (comme « npm test vert »). J'ai également rédigé la partie « Contexte » du `README.md` pour expliquer le besoin métier et définir les limites du projet.

> **EN CLAIR**
>
> - **`docs/taches.md`** : liste des tâches planifiées, écrite au format Markdown (`.md`), un format texte simple.
> - **Preuve de réussite** : critère vérifiable qui montre qu'une tâche est vraiment terminée. Exemple : « npm test vert », c'est-à-dire tous les tests automatiques réussis.
> - **`README.md`** : page de présentation du projet, la première lue par quelqu'un qui découvre le dépôt.



Capture 5 : *Le fichier docs/taches.md avec le tableau rempli.*



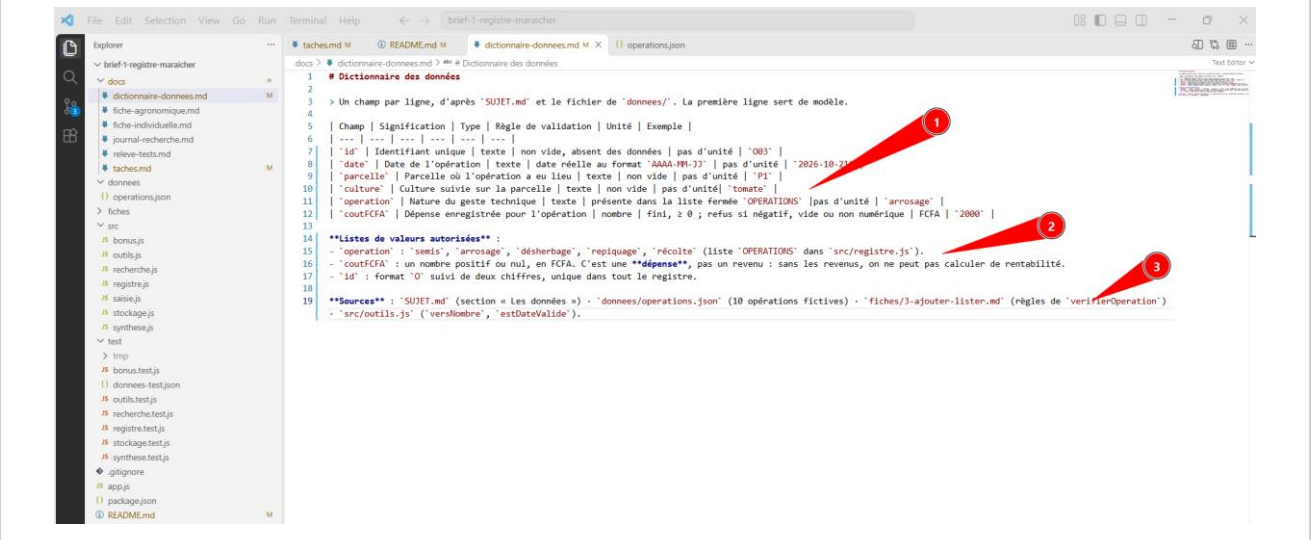
Capture 6 : *Le fichier README.md avec la partie Contexte.*

3. Conception et modélisation des données

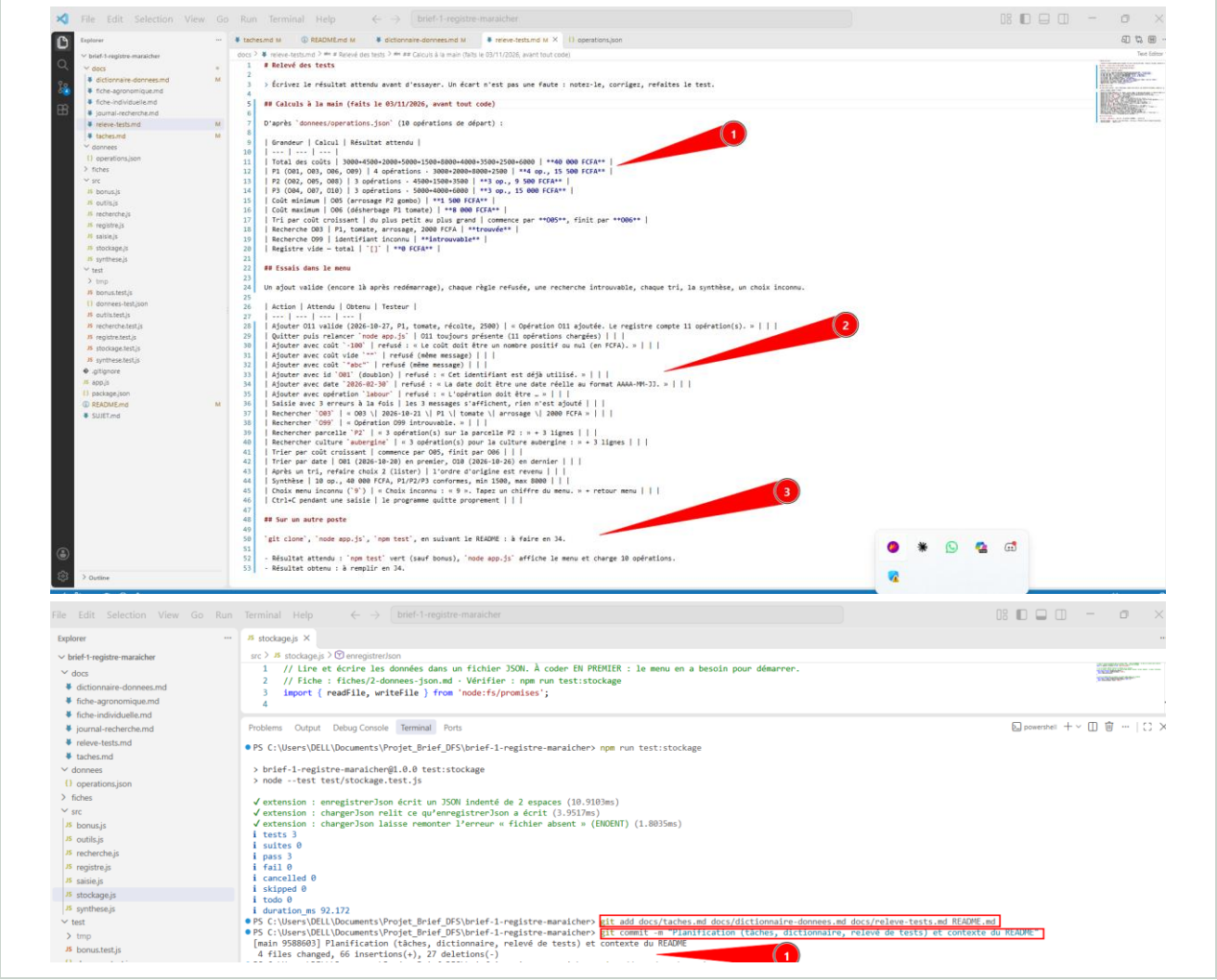
Avant de coder, j'ai défini la structure des données. J'ai rempli le `docs/dictionnaire-donnees.md` qui décrit les 6 champs du fichier JSON (`id`, `date`, `parcelle`, `culture`, `operation`, `coutFCFA`) et leurs règles de validation. J'ai aussi fait les calculs à la main dans `docs/releve-tests.md` (total de 40 000 FCFA, P1 à 15 500 FCFA, etc.) pour avoir une référence de comparaison.

EN CLAIR

- **Dictionnaire de données** : fiche qui décrit chaque champ (nom, rôle, règles de validation) pour que tout le monde comprenne la même chose.
- **Les 6 champs** : `id` (identifiant unique), `date`, `parcelle`, `culture`, `operation` (type d'opération) et `coutFCFA` (coût en francs CFA).
- **Relevé de tests fait à la main** : valeurs attendues calculées sans le programme. Une fois le code écrit, on compare : un écart révèle un bug.



Capture 7 : *Le fichier docs/dictionnaire-donnees.md.*



Capture 8 : *Le fichier docs/releve-tests.md.*

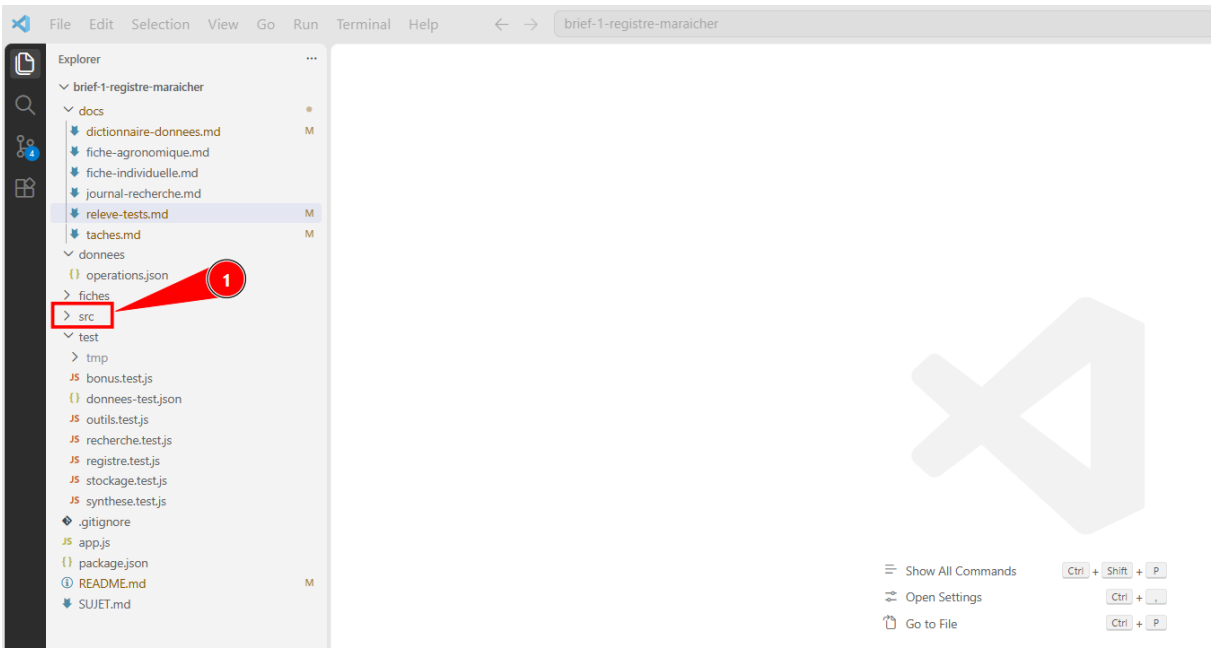
4. Développement du module de stockage (src/stockage.js)

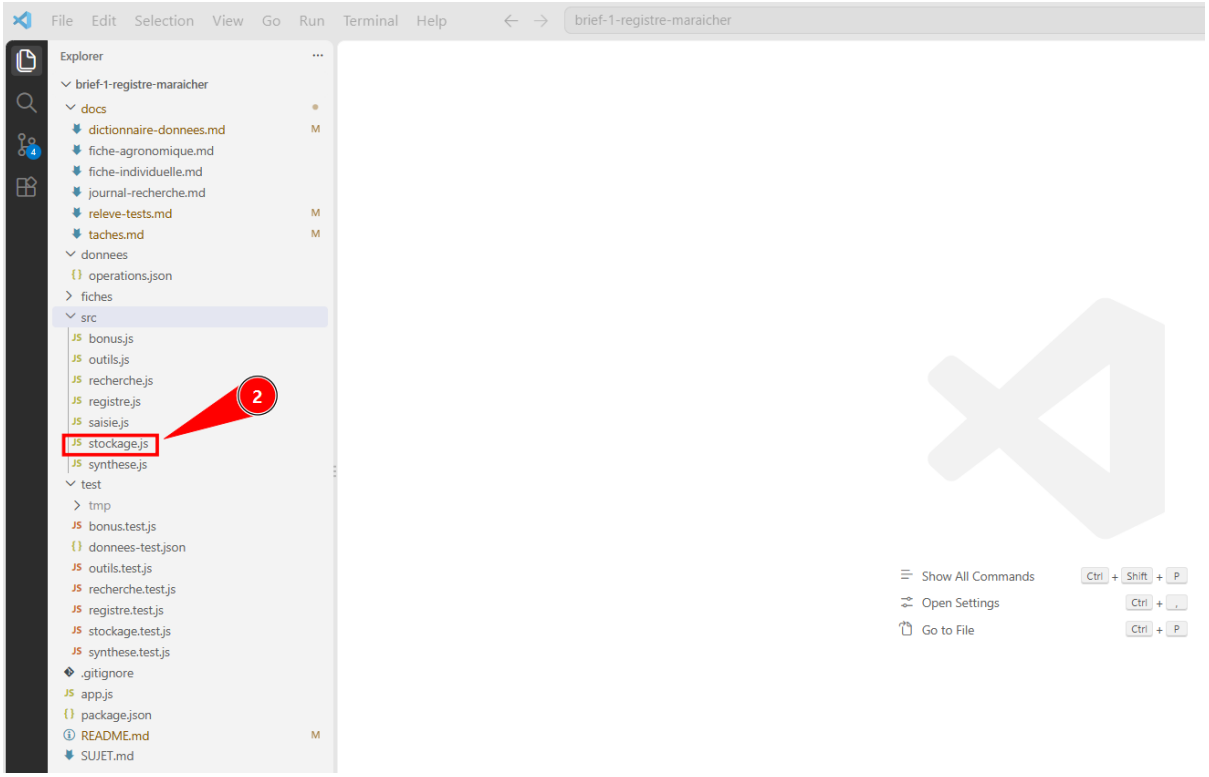
C'est le premier module que j'ai codé. Il lit et écrit dans le fichier `operations.json`.

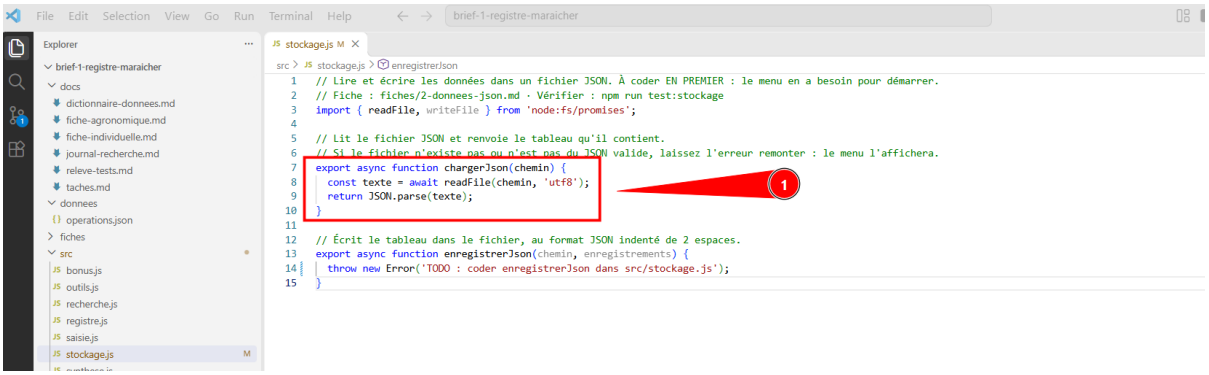
- **`chargerJson`** : Utilise `await readFile`. L'erreur est volontairement laissée remonter pour être gérée par le menu.
- **`enregistrerJson`** : Utilise `JSON.stringify(enregistrements, null, 2)`. Le 2 indente le fichier JSON pour qu'il reste lisible.

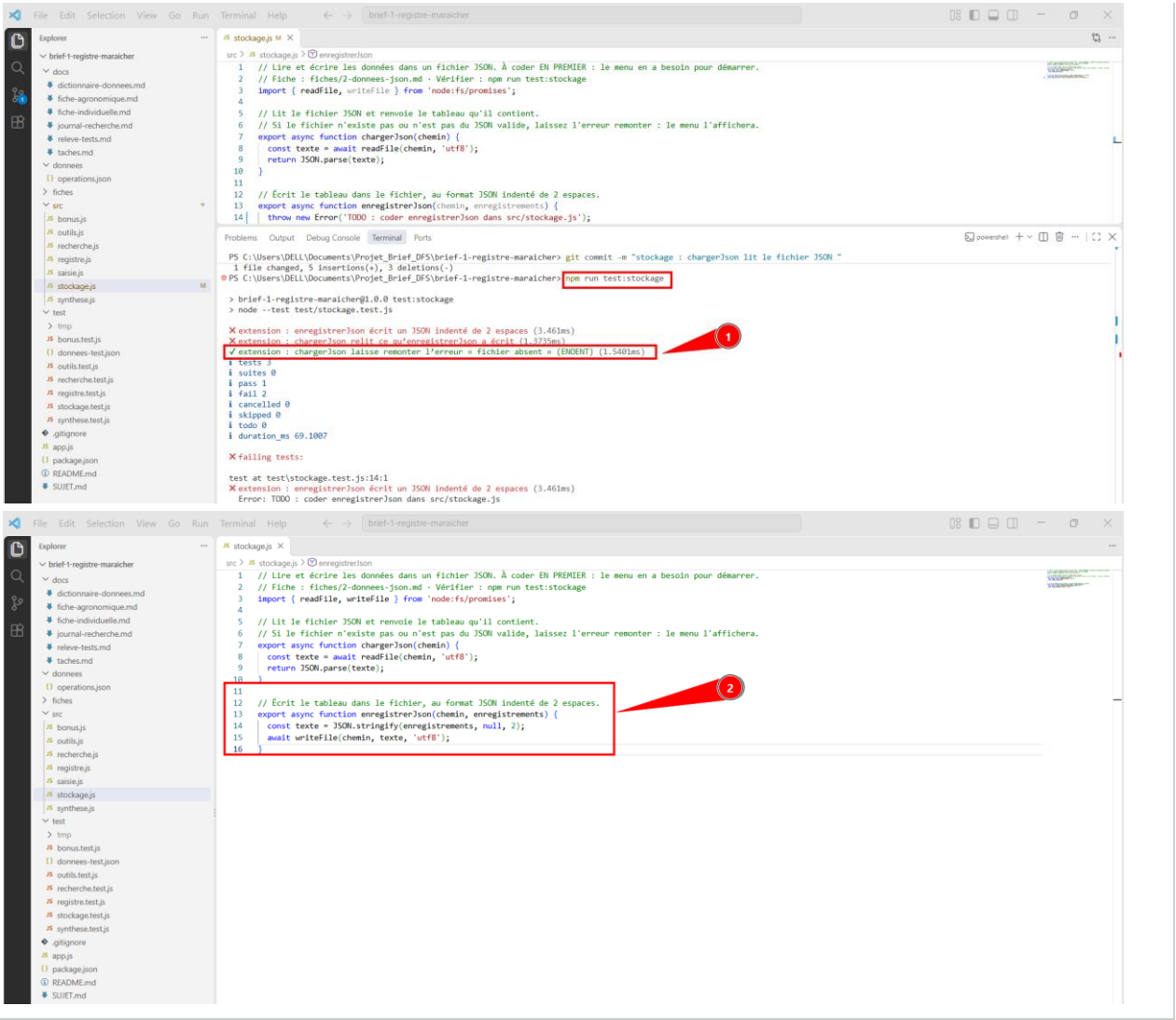
EN CLAIR

- **JSON** : format texte qui stocke les opérations sous forme de liste d'objets ; `operations.json` sert de mémoire à l'application.
- **`await readFile`** : lire un fichier prend du temps (opération asynchrone) ; `await` met le programme en attente jusqu'à la fin de la lecture.
- **Erreur « laissée remonter »** : `chargerJson` ne capture pas l'erreur (fichier introuvable, par exemple) : elle remonte jusqu'au menu (`app.js`), qui décide du message à afficher. Le module de stockage se contente de lire et d'écrire.
- **`JSON.stringify(x, null, 2)`** : transforme les données en texte JSON. `null` signifie « aucun filtre sur les champs » ; `2` demande une indentation de 2 espaces.
- **« 3 tests au vert »** : trois tests automatiques ont réussi.

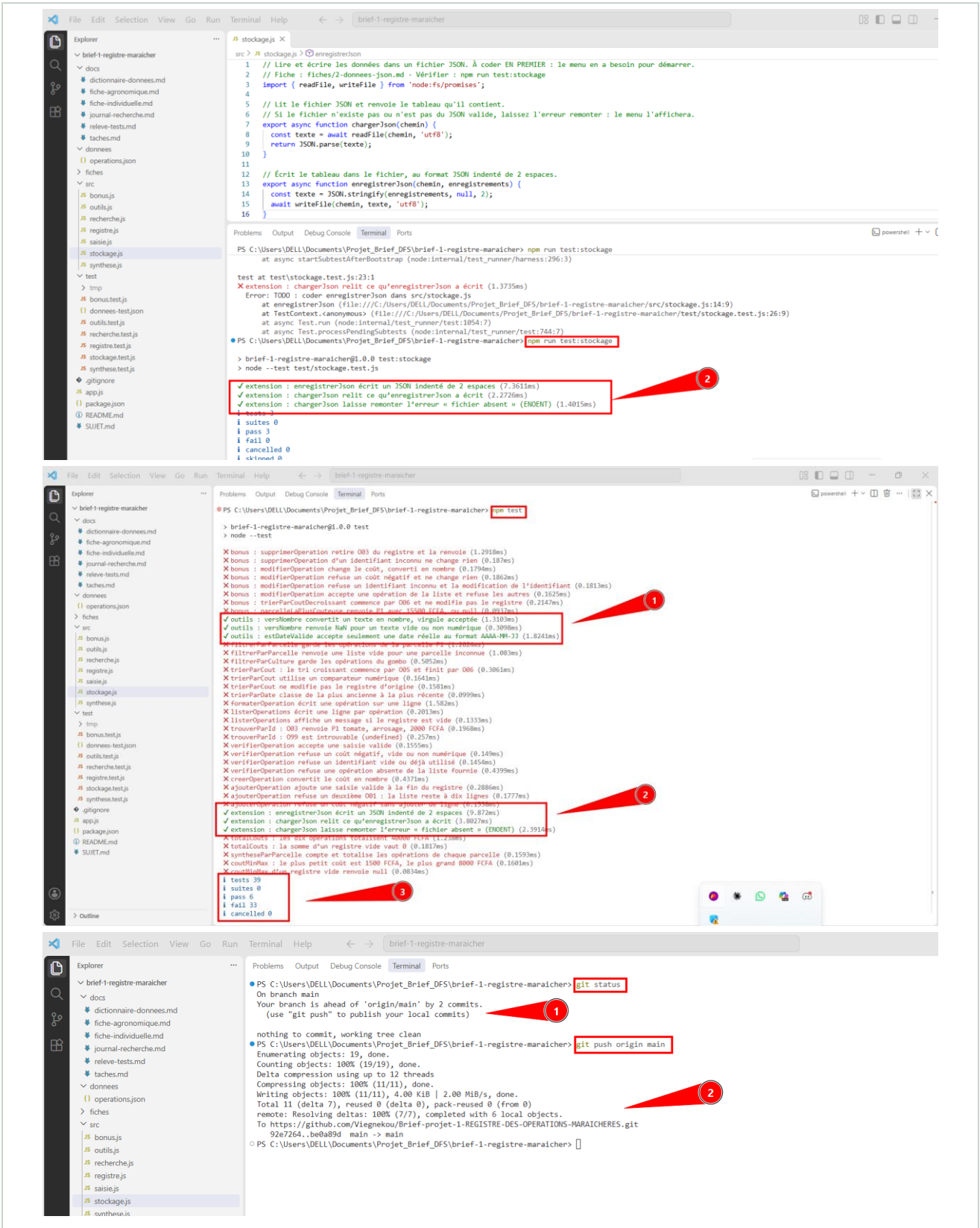

File Edit Selection View Go Run Terminal Help
brief-1-registre-maraicher
Explorer
brief-1-registre-maraicher
docs
dictionnaire-donnees.md
fiche-agronomique.md
fiche-individuelle.md
journal-recherche.md
releve-tests.md
taches.md
donnees
operations.json
fiches
src
test
tmp
bonus.test.js
donnees-test.json
outils.test.js
recherche.test.js
registre.test.js
stockage.test.js
synthese.test.js
.gitignore
app.js
package.json
README.md
SUJET.md
1
Show All Commands Ctrl + Shift + P
Open Settings Ctrl + ,
Go to File Ctrl + P


File Edit Selection View Go Run Terminal Help
brief-1-registre-maraicher
Explorer
brief-1-registre-maraicher
docs
dictionnaire-donnees.md
fiche-agronomique.md
fiche-individuelle.md
journal-recherche.md
releve-tests.md
taches.md
donnees
operations.json
fiches
src
bonus.js
outils.js
recherche.js
registre.js
saisie.js
stockage.js
synthese.js
test
tmp
bonus.test.js
donnees-test.json
outils.test.js
recherche.test.js
registre.test.js
stockage.test.js
synthese.test.js
.gitignore
app.js
package.json
README.md
SUJET.md
2
Show All Commands Ctrl + Shift + P
Open Settings Ctrl + ,
Go to File Ctrl + P


File Edit Selection View Go Run Terminal Help
brief-1-registre-maraicher
Explorer
stockage.js M
src > stockage.js > enregistrerJson
// Lire et écrire les données dans un fichier JSON. À coder EN PREMIER : le menu en a besoin pour démarrer.
// Fiche : fiches/2-donnees-json.md · Vérifier : npm run test:stockage
import { readFile, writeFile } from 'node:fs/promises';
// Lit le fichier JSON et renvoie le tableau qu'il contient.
// Si le fichier n'existe pas ou n'est pas du JSON valide, laissez l'erreur remonter : le menu l'affichera.
export async function chargerJson(chemin) {
const texte = await readFile(chemin, 'utf8');
return JSON.parse(texte);
}
// Écrit le tableau dans le fichier, au format JSON indenté de 2 espaces.
export async function enregistrerJson(chemin, enregistrements) {
throw new Error('TODO : coder enregistrerJson dans src/stockage.js');
}
1



Capture 9 : *Le code de src/stockage.js.*



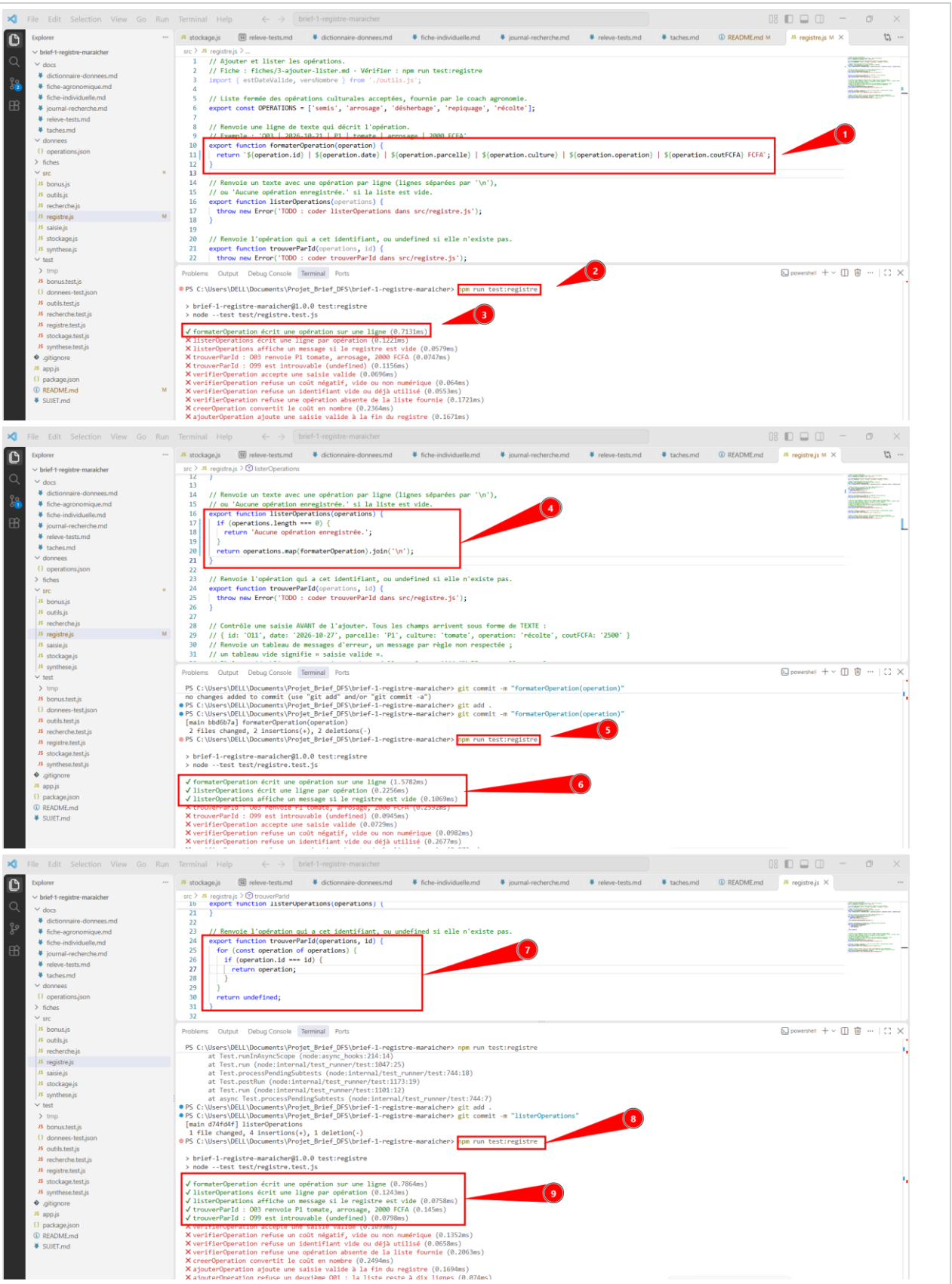
Capture 10 : *Le terminal avec npm run test:stockage et les 3 tests au vert.*

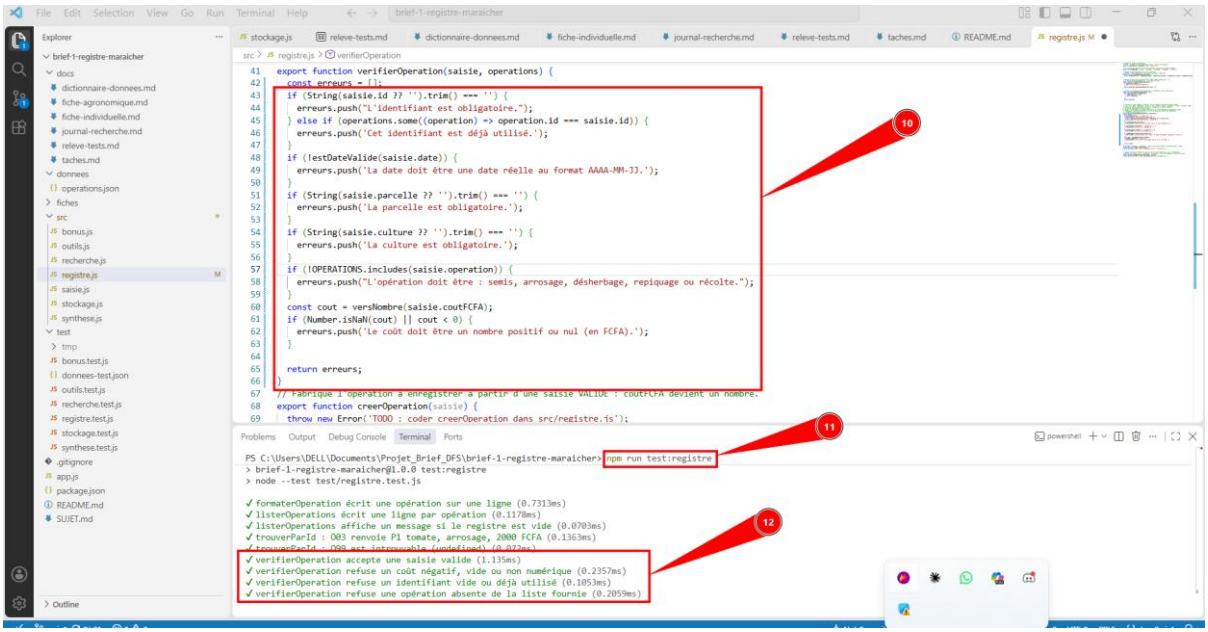
5. Développement du registre et des règles métier (src/registre.js)

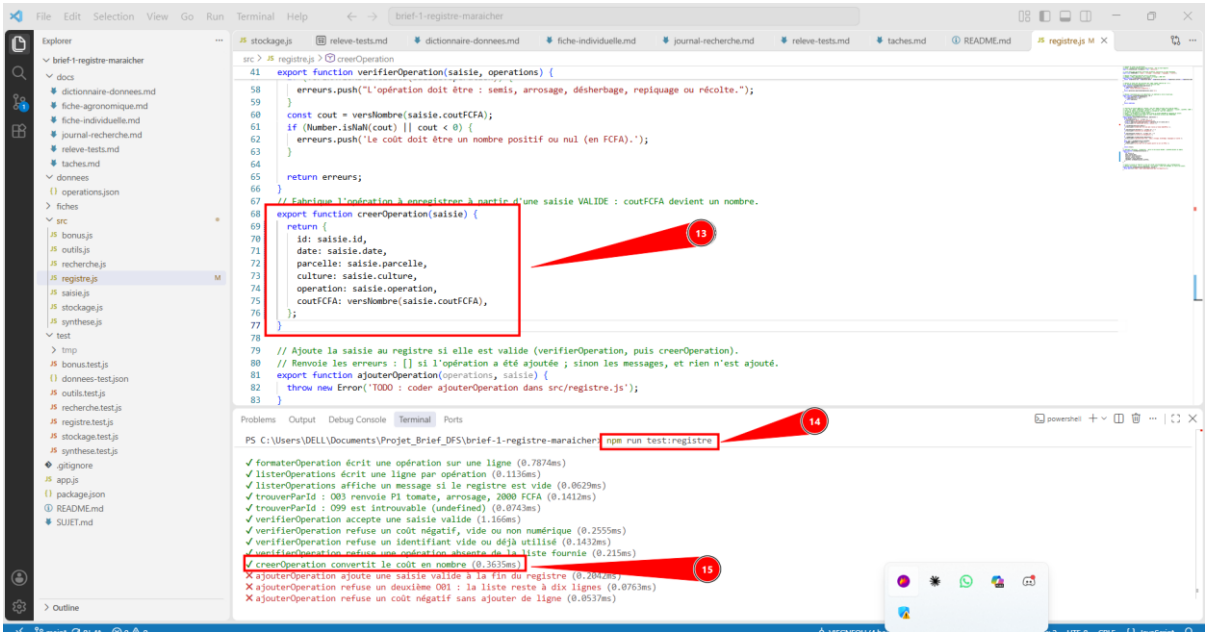
Ce fichier contient la logique de vérification.

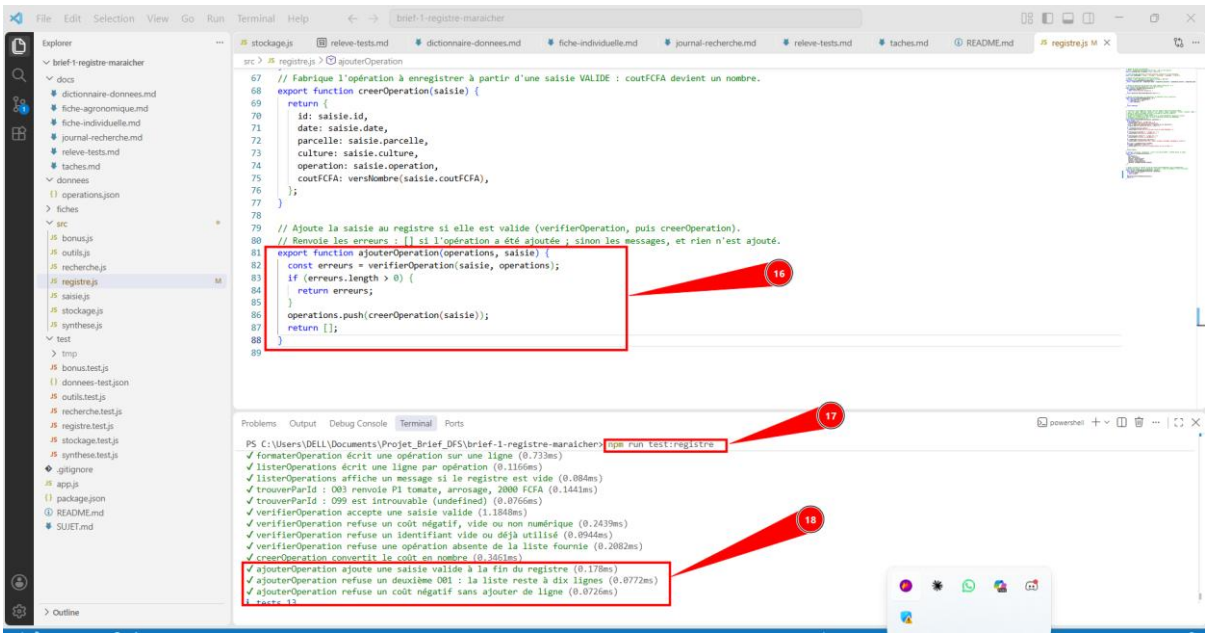
- **`verifierOperation`** : Vérifie que l'id est unique, la date valide, l'opération dans la liste et le coût positif (via `versNombre`). Elle accumule les erreurs dans un tableau.
- **`ajouterOperation`** : Utilise `push()` pour modifier le tableau principal.
- **`formaterOperation`** : Formate l'affichage avec des séparateurs `|`.

> **EN CLAIR**
>
> - **Accumuler les erreurs** : au lieu de s'arrêter à la première anomalie, `verifierOperation` les range toutes dans un tableau : l'utilisateur voit tous ses problèmes d'un coup.
> - **`versNombre`** : une saisie au clavier est du texte (« 15000 ») ; pour vérifier que le coût est positif, il faut d'abord le convertir en nombre.
> - **Id unique** : deux opérations ne peuvent pas partager le même identifiant, sinon on ne saurait plus laquelle modifier ou supprimer.
> - **`push()`** : ajoute un élément directement dans le tableau d'origine (on parle de mutation). C'est voulu ici : c'est bien le registre principal qu'il faut mettre à jour.

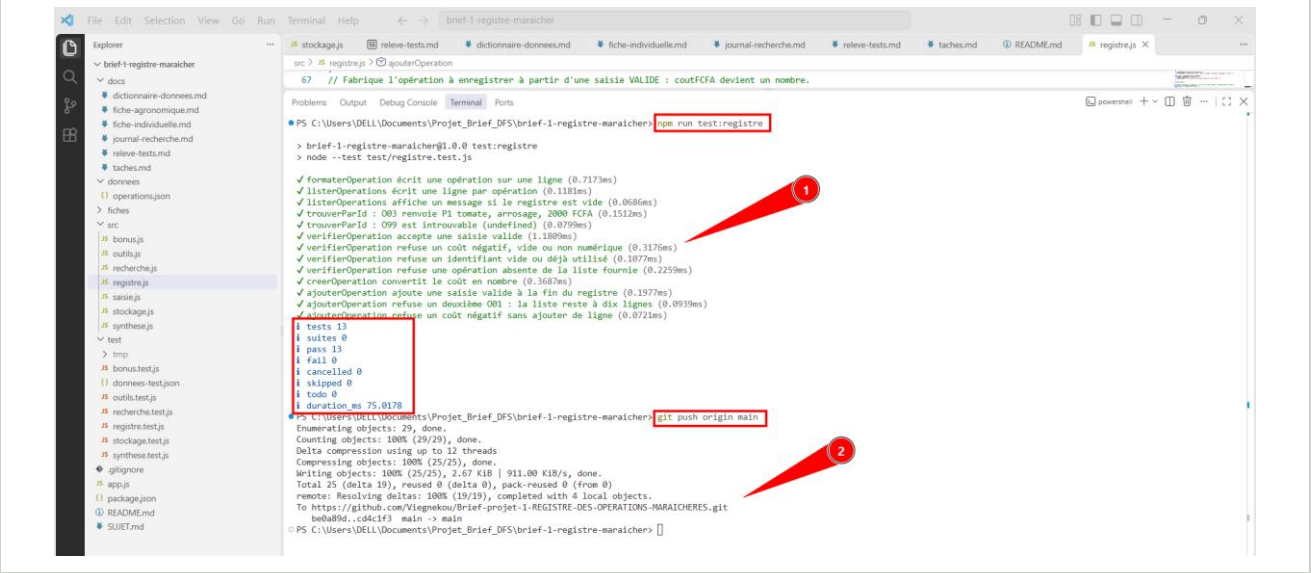

1
2
3
4
5
6
7
8
9


export function verifierOperation(saisie, operations) {
const erreurs = [];
if (String(saisie.id ?? '').trim() === '') {
erreurs.push("L'identifiant est obligatoire.");
} else if (operations.some((operation) => operation.id === saisie.id)) {
erreurs.push('Cet identifiant est déjà utilisé.');
}
if (!estDateValide(saisie.date)) {
erreurs.push('La date doit être une date réelle au format AAAA-MM-JJ.');
}
if (String(saisie.parcelle ?? '').trim() === '') {
erreurs.push('La parcelle est obligatoire.');
}
if (String(saisie.culture ?? '').trim() === '') {
erreurs.push('La culture est obligatoire.');
}
if (!OPERATIONS.includes(saisie.operation)) {
erreurs.push("L'opération doit être : semis, arrosage, désherbage, repiquage ou récolte.");
}
const cout = versNombre(saisie.coutFCFA);
if (Number.isNaN(cout) || cout < 0) {
erreurs.push('Le coût doit être un nombre positif ou nul (en FCFA).');
}
return erreurs;
}
// Fabrique l'opération à enregistrer à partir d'une saisie VALIDE : coutFCFA devient un nombre.
export function creerOperation(saisie) {
throw new Error('TODO : coder creerOperation dans src/registre.js');
10
11
PS C:\Users\DELL\Documents\Projet_Brief_DFS\brief-1-registre-maraicher> npm run test:registre
> brief-1-registre-maraicher@1.0.0 test:registre
> node --test test/registre.test.js
✓ formaterOperation écrit une opération sur une ligne (0.7313ms)
✓ listerOperations écrit une ligne par opération (0.1178ms)
✓ listerOperations affiche un message si le registre est vide (0.0703ms)
✓ trouverParId : 003 renvoie P1 tomate, arrosage, 2000 FCFA (0.1363ms)
✓ trouverParId : 099 est introuvable (undefined) (0.072ms)
12
✓ verifierOperation accepte une saisie valide (1.135ms)
✓ verifierOperation refuse un coût négatif, vide ou non numérique (0.2357ms)
✓ verifierOperation refuse un identifiant vide ou déjà utilisé (0.1053ms)
✓ verifierOperation refuse une opération absente de la liste fournie (0.2059ms)


13
14
PS C:\Users\DELL\Documents\Projet_Brief_DFS\brief-1-registre-maraicher> npm run test:registre
✓ formaterOperation écrit une opération sur une ligne (0.7874ms)
✓ listerOperations écrit une ligne par opération (0.1136ms)
✓ listerOperations affiche un message si le registre est vide (0.0629ms)
✓ trouverParId : 003 renvoie P1 tomate, arrosage, 2000 FCFA (0.1412ms)
✓ trouverParId : 099 est introuvable (undefined) (0.0743ms)
✓ verifierOperation accepte une saisie valide (1.166ms)
✓ verifierOperation refuse un coût négatif, vide ou non numérique (0.2555ms)
✓ verifierOperation refuse un identifiant vide ou déjà utilisé (0.1432ms)
✓ verifierOperation refuse une opération absente de la liste fournie (0.215ms)
15
✓ creerOperation convertit le coût en nombre (0.3635ms)
✖ ajouterOperation ajoute une saisie valide à la fin du registre (0.2042ms)
✖ ajouterOperation refuse un deuxième 001 : la liste reste à dix lignes (0.0763ms)
✖ ajouterOperation refuse un coût négatif sans ajouter de ligne (0.0537ms)


// Fabrique l'opération à enregistrer à partir d'une saisie VALIDE : coutFCFA devient un nombre.
export function creerOperation(saisie) {
return {
id: saisie.id,
date: saisie.date,
parcelle: saisie.parcelle,
culture: saisie.culture,
operation: saisie.operation,
coutFCFA: versNombre(saisie.coutFCFA),
};
}
// Ajoute la saisie au registre si elle est valide (verifierOperation, puis creerOperation).
// Renvoie les erreurs : [] si l'opération a été ajoutée ; sinon les messages, et rien n'est ajouté.
export function ajouterOperation(operations, saisie) {
const erreurs = verifierOperation(saisie, operations);
if (erreurs.length > 0) {
return erreurs;
}
operations.push(creerOperation(saisie));
return [];
}
16
17
PS C:\Users\DELL\Documents\Projet_Brief_DFS\brief-1-registre-maraicher> npm run test:registre
✓ formaterOperation écrit une opération sur une ligne (0.733ms)
✓ listerOperations écrit une ligne par opération (0.1166ms)
✓ listerOperations affiche un message si le registre est vide (0.084ms)
✓ trouverParId : 003 renvoie P1 tomate, arrosage, 2000 FCFA (0.1441ms)
✓ trouverParId : 099 est introuvable (undefined) (0.0766ms)
✓ verifierOperation accepte une saisie valide (1.1848ms)
✓ verifierOperation refuse un coût négatif, vide ou non numérique (0.2439ms)
✓ verifierOperation refuse un identifiant vide ou déjà utilisé (0.0944ms)
✓ verifierOperation refuse une opération absente de la liste fournie (0.2082ms)
✓ creerOperation convertit le coût en nombre (0.3451ms)
18
✓ ajouterOperation ajoute une saisie valide à la fin du registre (0.178ms)
✓ ajouterOperation refuse un deuxième 001 : la liste reste à dix lignes (0.0772ms)
✓ ajouterOperation refuse un coût négatif sans ajouter de ligne (0.0726ms)

Capture 11 : Le code de src/registre.js (en particulier verifierOperation).



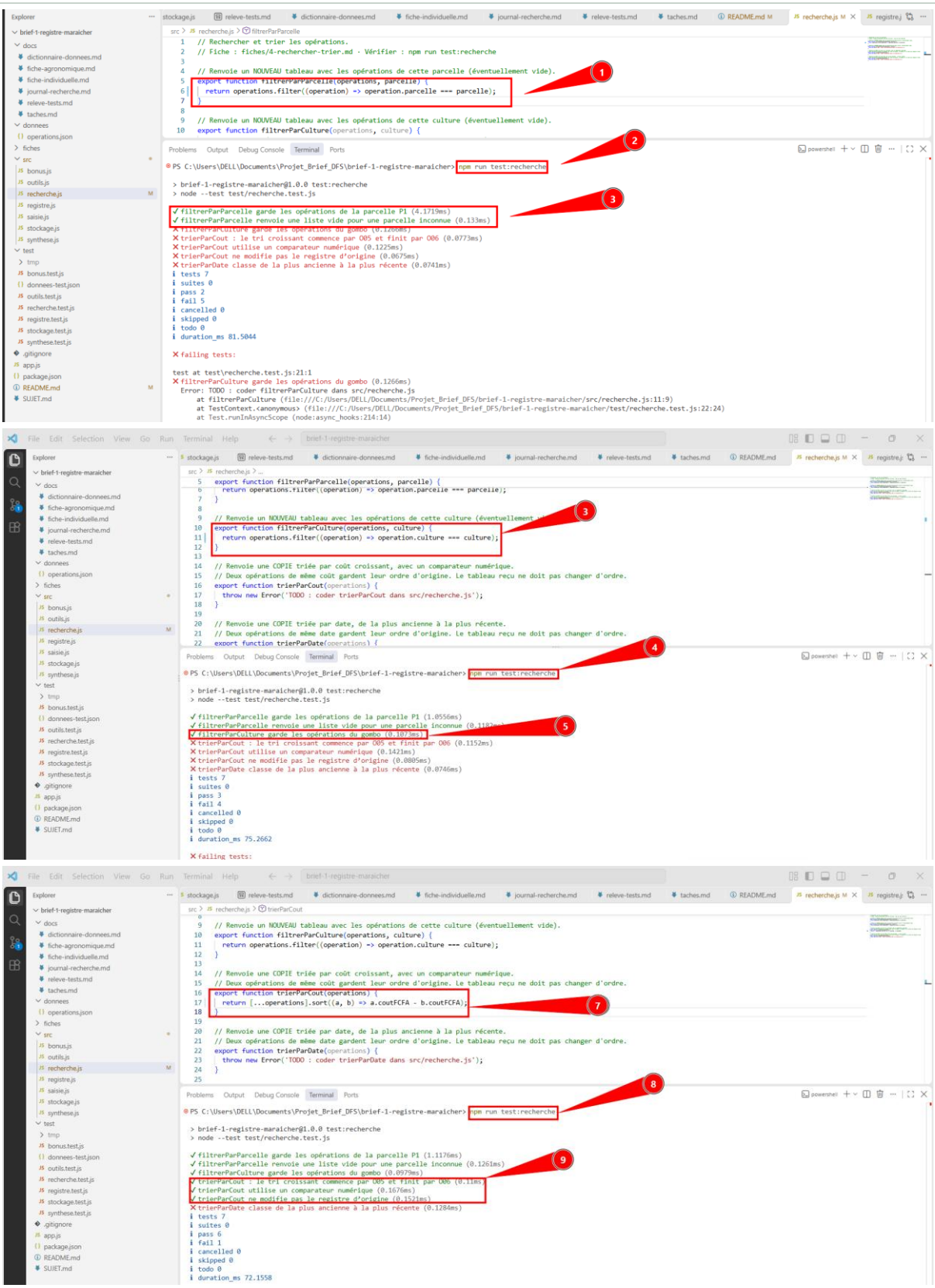
Capture 12 : Le terminal avec npm run test:registre et les 13 tests au vert.

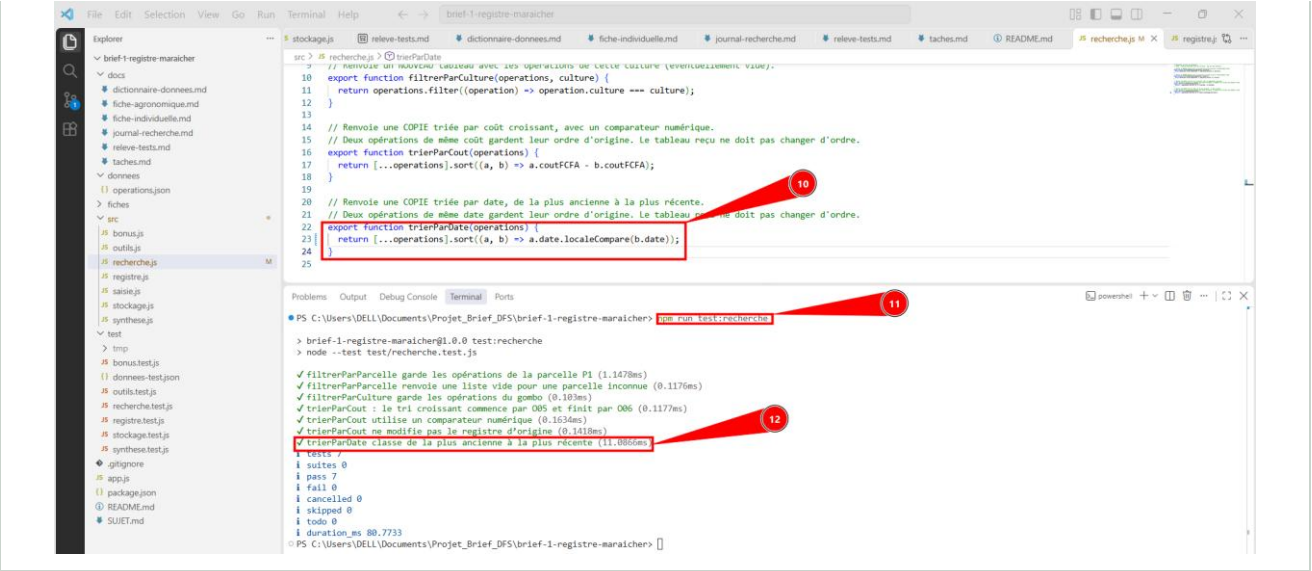
6. Développement de la recherche et du tri (src/recherche.js)

Ce module filtre et trie les données.

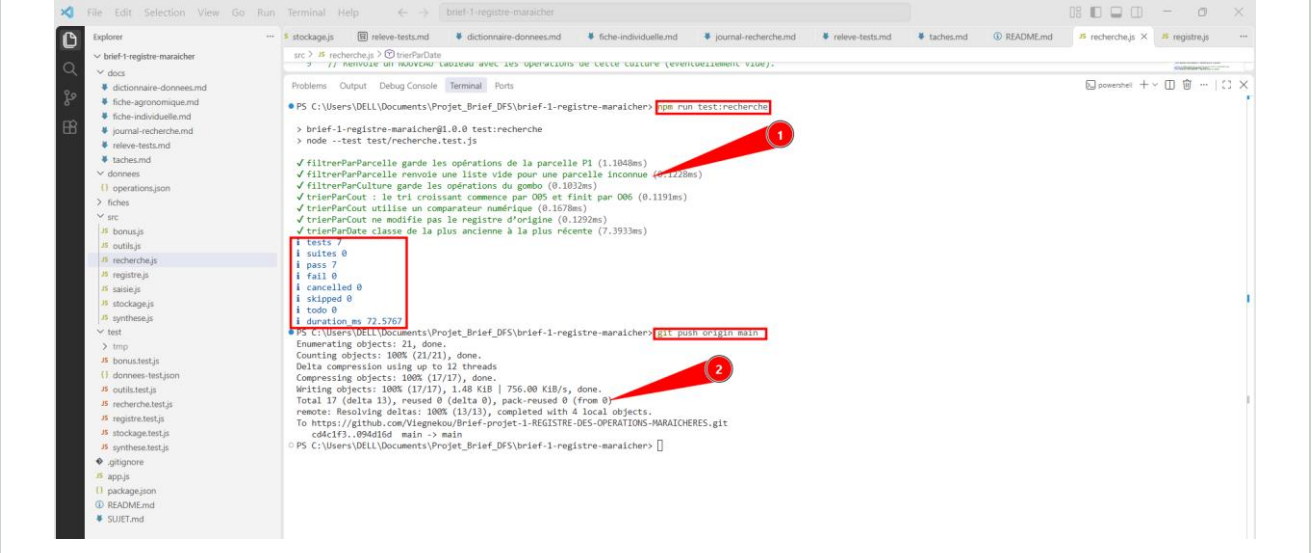
- **`Filtres`** : Utilisation de `filter()` pour créer un nouveau tableau.
- **`Tris`** : Utilisation de `[...operations].sort(...)`. Le piège évité : `sort()` modifie le tableau d'origine. La copie `[...operations]` protège l'ordre du registre principal. Le comparateur `(a, b) => a.coutFCFA - b.coutFCFA` évite le tri alphabétique.

> **EN CLAIR**
>
> - **`filter()`** : renvoie un nouveau tableau contenant uniquement les éléments qui respectent la condition ; le tableau d'origine n'est pas touché.
> - **Le piège de `sort()`** : il réordonne le tableau d'origine. `[...operations]` fabrique d'abord une copie, que l'on trie à la place : le registre garde son ordre.
> - **Le comparateur** : sans lui, `sort()` compare du texte : `[9, 100, 25].sort()` donne `[100, 25, 9]`. Avec `(a, b) => a.coutFCFA - b.coutFCFA`, un résultat négatif place `a` avant `b`, un résultat positif après : l'ordre devient numérique.


1
2
3
4
5
7
8
9



Capture 13 : *Le code de src/recherche.js.*



Capture 14 : *Le terminal avec npm run test:recherche et les 7 tests au vert.*

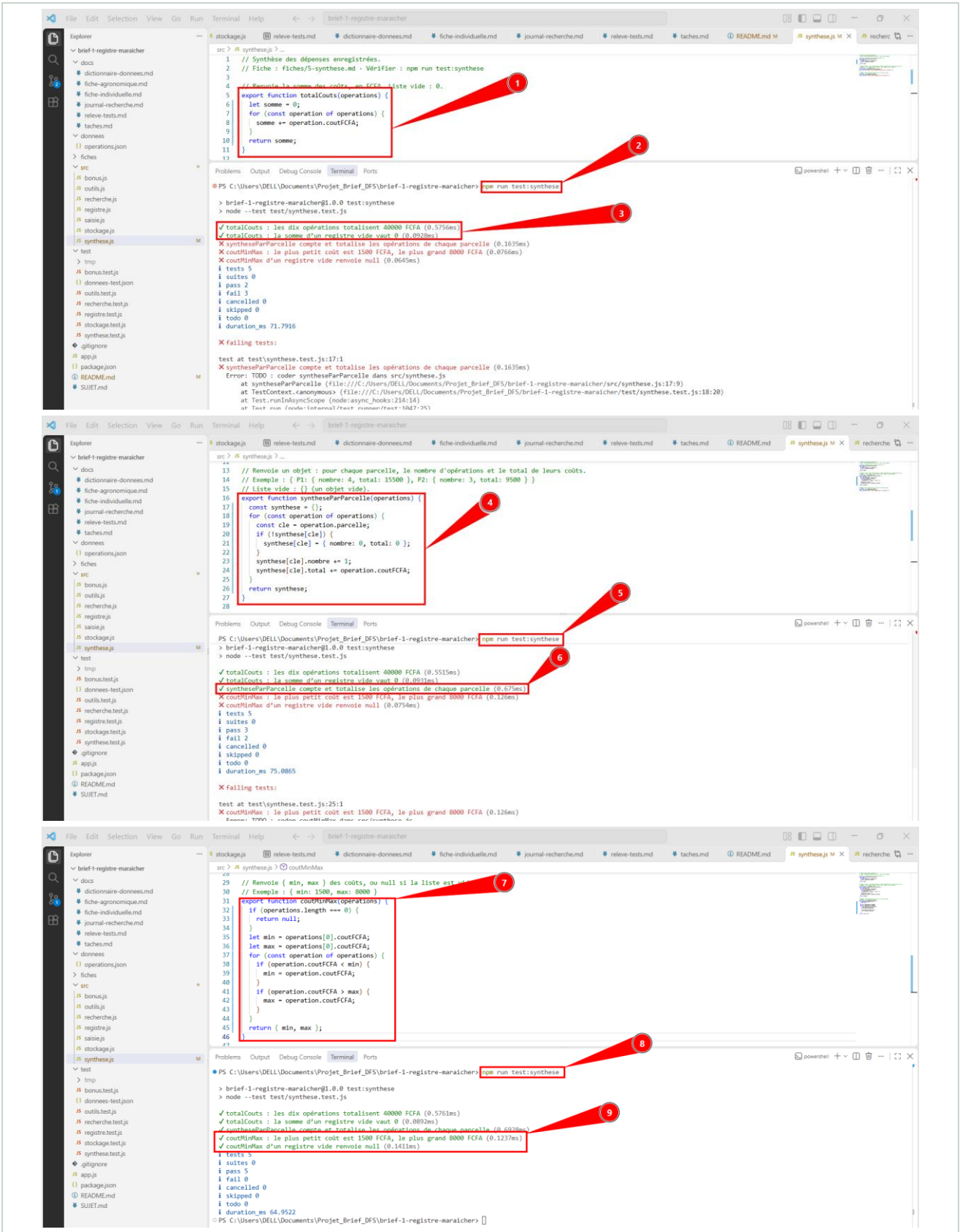
7. Développement de la synthèse (src/synthese.js)

Ce module fait les calculs.

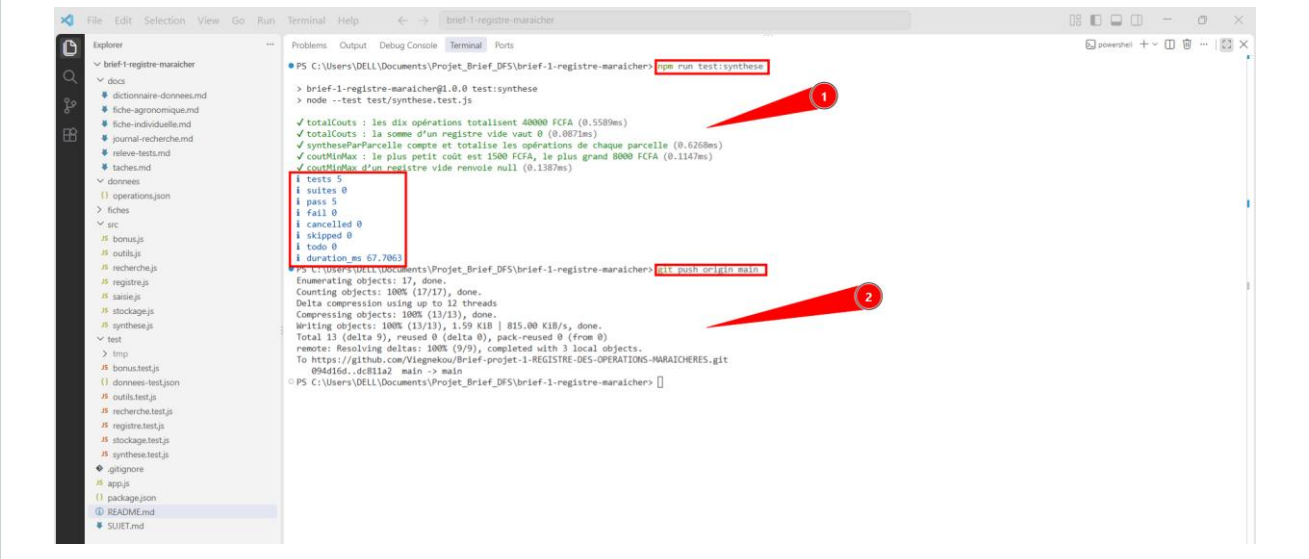
- **`totalCouts`** : Utilise `.reduce()` avec une valeur initiale à 0 pour gérer le cas du tableau vide.
- **`syntheseParParcelle`** : Regroupe les totaux dans un objet.
- **`coutMinMax`** : Le piège évité : `Math.min()` sur un tableau vide renvoie `Infinity`. J'ai ajouté une condition `if (operations.length === 0) return null`.

EN CLAIR

- `.reduce()` **avec valeur initiale 0** : parcourt le tableau en cumulant un total. Avec 0 au départ, un tableau vide donne 0 ; sans valeur initiale, JavaScript déclenche une erreur sur un tableau vide.
- `syntheseParParcelle` : renvoie un objet dont chaque clé est une parcelle et chaque valeur son total, par exemple `{ P1: 15500, … }`.
- `Math.min()` **sur un tableau vide** : renvoie `Infinity`, un résultat absurde (`Math.max()` renvoie `-Infinity`). La condition renvoie `null`, qui signifie « pas de valeur », et le menu peut alors afficher un message clair.



Capture 15 : *Le code de src/synthese.js.*



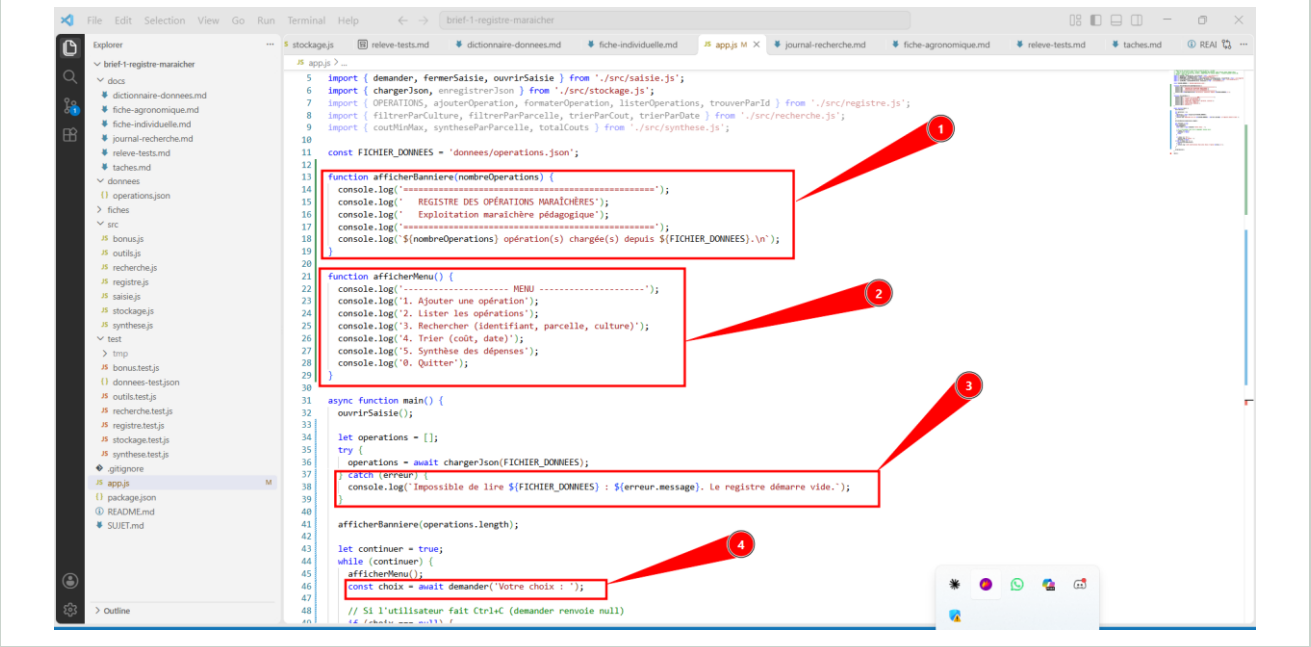
Capture 16 : *Le terminal avec npm run test:synthese et les 5 tests au vert.*

8. Développement du menu principal (app.js)

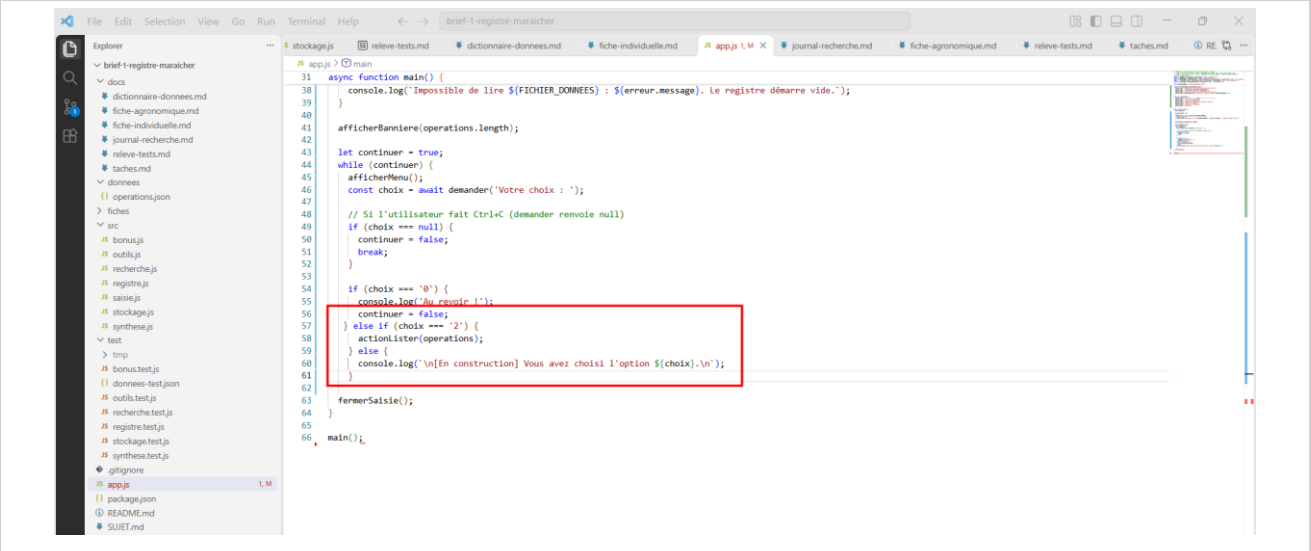
C'est le chef d'orchestre.

- **`Boucle while`** : pour l'affichage continu du menu.
- **`Majuscules / minuscules`** : gestion de la casse (`toUpperCase()`, `toLowerCase()`).
- **`Gestion des erreurs`** : un `try/catch` englobe le menu pour éviter les plantages. Le cas Ctrl+C (qui renvoie `null`) est géré pour quitter proprement.

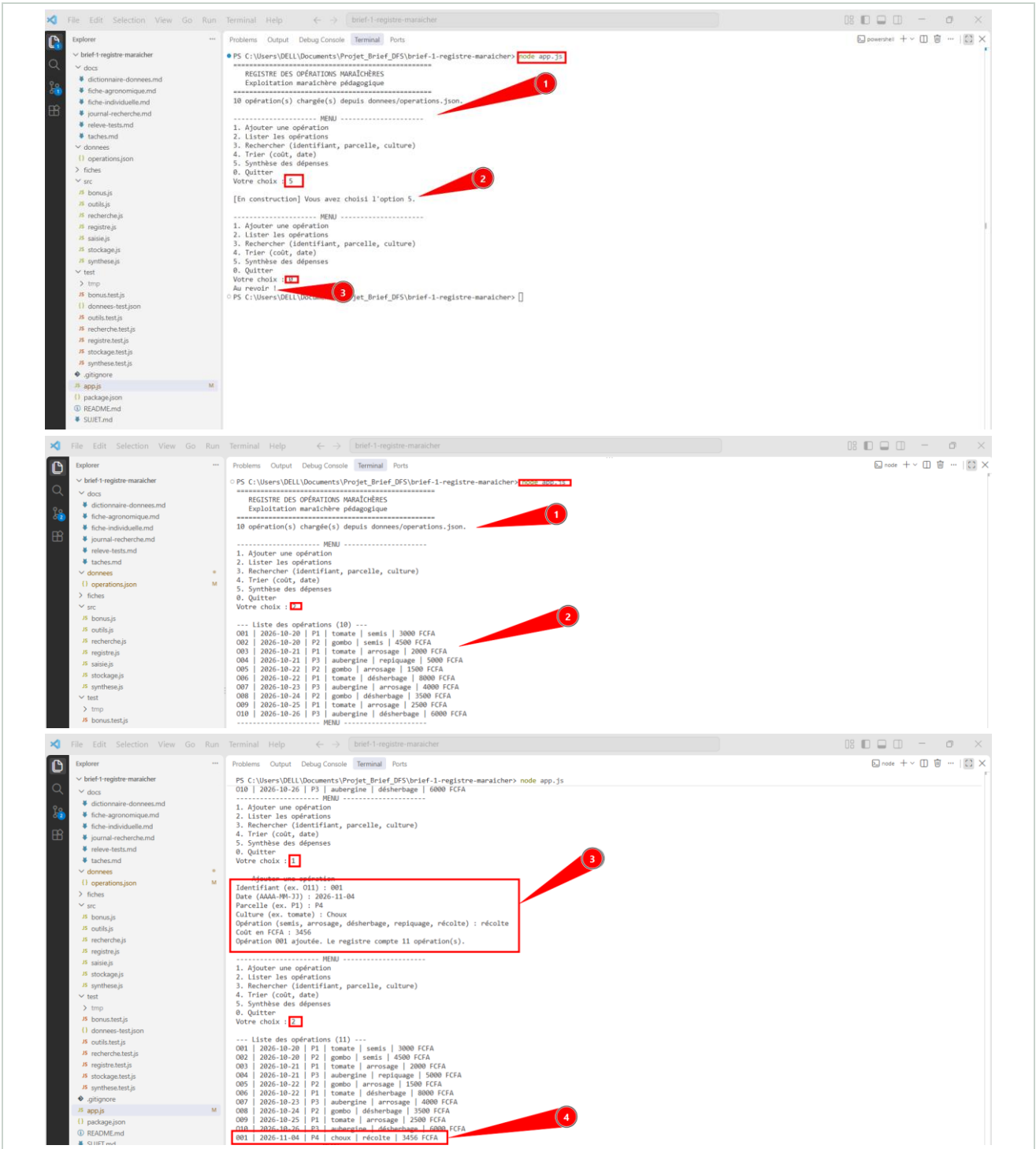
> **EN CLAIR**
>
> - **« Chef d'orchestre »** : `app.js` ne calcule rien lui-même : il appelle les autres modules (stockage, registre, recherche, synthèse) selon le choix fait dans le menu.
> - **Boucle `while`** : réaffiche le menu en continu jusqu'à ce que l'utilisateur choisisse de quitter.
> - **`toUpperCase()` / `toLowerCase()`** : convertissent la saisie en majuscules ou minuscules avant la comparaison, pour que « q » et « Q » donnent le même résultat.
> - **`try/catch`** : si une erreur survient dans le bloc `try`, l'exécution saute au bloc `catch`, où l'on affiche un message au lieu de laisser le programme planter.
> - **Ctrl+C et `null`** : quand l'utilisateur interrompt la saisie, la valeur reçue est `null`. Sans vérification, le programme planterait en l'utilisant comme du texte ; en la gérant, il quitte proprement.



Capture 17 : *Le code de app.js (les imports et le menu).*

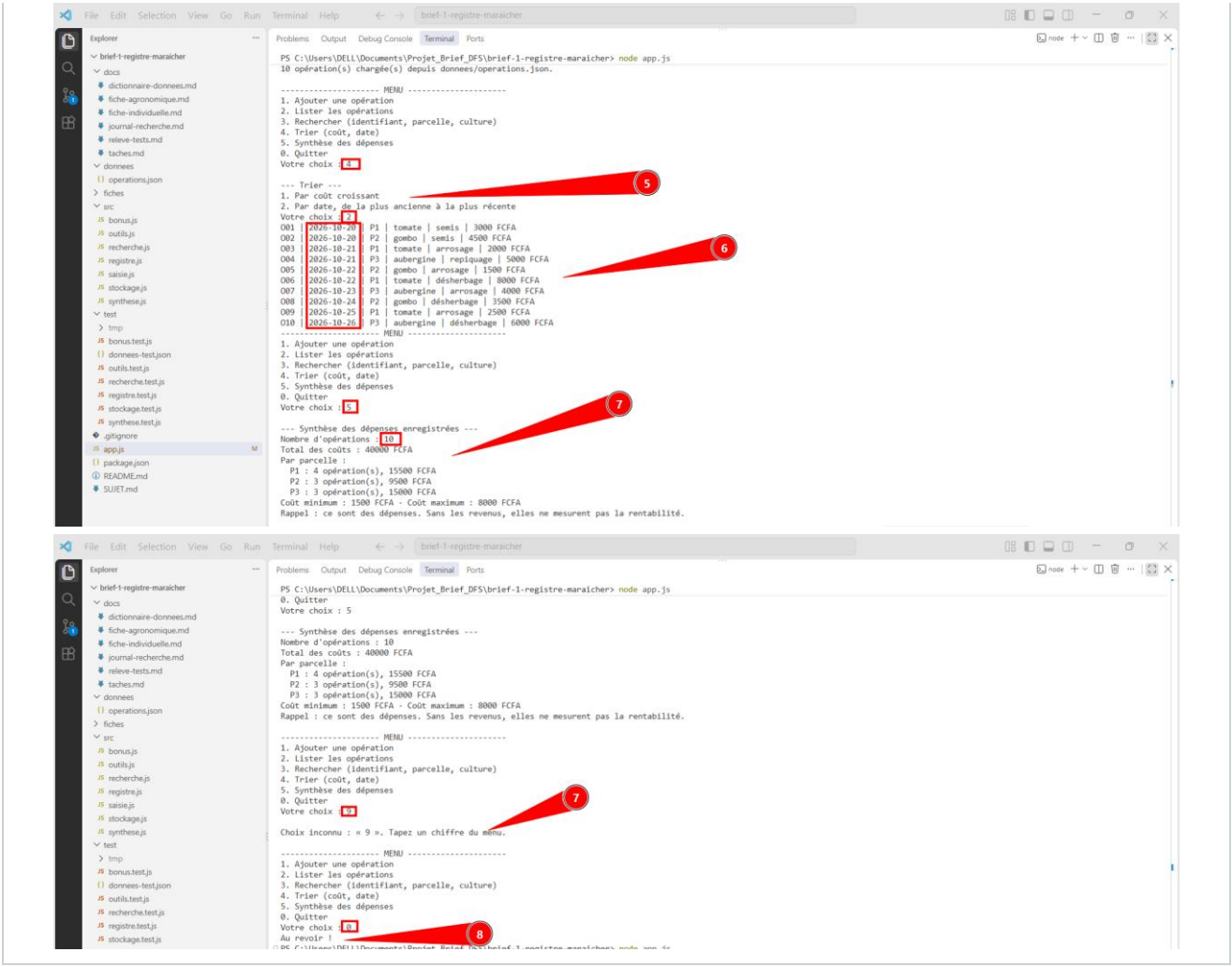


Capture 18 : *Le code de app.js (la boucle et le try/catch).*


PS C:\Users\DELL\Documents\Projet_Brief_DFS\brief-1-registre-maraicher> node app.js
REGISTRE DES OPÉRATIONS MARAÎCHÈRES
Exploitation maraîchère pédagogique
10 opération(s) chargée(s) depuis donnees/operations.json.
---------- MENU ----------
1. Ajouter une opération
2. Lister les opérations
3. Rechercher (identifiant, parcelle, culture)
4. Trier (coût, date)
5. Synthèse des dépenses
0. Quitter
Votre choix : 5
[En construction] Vous avez choisi l'option 5.
Votre choix : 0
Au revoir !
1
2
3

REGISTRE DES OPÉRATIONS MARAÎCHÈRES
Exploitation maraîchère pédagogique
10 opération(s) chargée(s) depuis donnees/operations.json.
Votre choix : 2
--- Liste des opérations (10) ---
001 | 2026-10-20 | P1 | tomate | semis | 3000 FCFA
002 | 2026-10-20 | P2 | gombo | semis | 4500 FCFA
003 | 2026-10-21 | P1 | tomate | arrosage | 2000 FCFA
004 | 2026-10-21 | P3 | aubergine | repiquage | 5000 FCFA
005 | 2026-10-22 | P2 | gombo | arrosage | 1500 FCFA
006 | 2026-10-22 | P1 | tomate | désherbage | 8000 FCFA
007 | 2026-10-23 | P3 | aubergine | arrosage | 4000 FCFA
008 | 2026-10-24 | P2 | gombo | désherbage | 3500 FCFA
009 | 2026-10-25 | P1 | tomate | arrosage | 2500 FCFA
010 | 2026-10-26 | P3 | aubergine | désherbage | 6000 FCFA
1
2

Identifiant (ex. O11) : 001
Date (AAAA-MM-JJ) : 2026-11-04
Parcelle (ex. P1) : P4
Culture (ex. tomate) : Choux
Opération (semis, arrosage, désherbage, repiquage, récolte) : récolte
Coût en FCFA : 3456
Opération 001 ajoutée. Le registre compte 11 opération(s).
001 | 2026-11-04 | P4 | choux | récolte | 3456 FCFA
3
4



Capture 19 : *Le terminal avec node app.js et le menu affiché.*

9. Développement du bonus (src/bonus.js et app.js)

Ajout de fonctionnalités supplémentaires.

- **`supprimerOperation`** : Utilise `findIndex()` et `splice()`.
- **`modifierOperation`** : Réutilise `verifierOperation`. Le piège évité : j'ai filtré le tableau pour exclure l'id en cours de modification afin d'éviter les faux doublons.
- **`parcelleLaPlusCouteuse`** : Parcourt la synthèse pour trouver le total le plus élevé.

EN CLAIR

- **Bonus** : fonctionnalités ajoutées en plus de celles demandées par le brief (ici, les choix 6 et 7 du menu).
- **`findIndex()` + `splice()`** : `findIndex()` donne la position de l'opération dans le tableau (−1 si elle n'existe pas) ; `splice(position, 1)` retire cet élément.
- **Le « faux doublon »** : lors d'une modification, l'opération existe déjà avec son id. Si on la compare à tout le registre, elle se détecte elle-même comme doublon. On exclut donc l'opération en cours du tableau avant d'appeler `verifierOperation`.
- **`parcelleLaPlusCouteuse`** : s'appuie sur la synthèse par parcelle et retient celle dont le total est le plus élevé.

10. Bilan final et tests de bout en bout

Vérification finale du programme complet. J'ai lancé tous les tests et testé les scénarios de refus (coût négatif, doublon) pour m'assurer que le programme affiche les bons messages sans planter.

EN CLAIR

- **Test de bout en bout** : on utilise le programme complet comme un vrai utilisateur, du menu jusqu'au fichier JSON, au lieu de tester chaque fonction isolément.
- **Scénario de refus** : cas volontairement invalide (coût négatif, id en doublon) pour vérifier que le programme refuse la saisie avec un message clair, sans planter.
- **`npm test`** : lance l'ensemble des tests automatiques en une seule commande.

Conclusion

Ce projet m'a permis de mettre en pratique des notions avancées de JavaScript (manipulation de tableaux, objets, asynchrone) et d'apprendre à structurer un projet de manière professionnelle (versioning avec Git, tests unitaires, documentation). J'ai également compris l'importance de séparer les responsabilités (chaque fichier a un rôle précis) et de gérer les erreurs pour offrir une expérience utilisateur robuste.

Annexe — Glossaire

Les termes techniques utilisés dans ce rapport, expliqués simplement.

Terme	Définition
Asynchrone / `await`	Une opération asynchrone (comme lire un fichier) prend du temps. `await` met le programme en pause à cet endroit jusqu'à ce qu'elle soit terminée.
Commit	Point de sauvegarde enregistré dans Git, accompagné d'un message qui décrit ce qui a changé.
Dépôt (repository)	Dossier de projet suivi par Git, avec tout son historique. Il peut être local (sur l'ordinateur) ou distant (sur GitHub).
FCFA	Franc CFA, monnaie dans laquelle sont exprimés les coûts des opérations.
Git / GitHub	Git garde l'historique des versions du code ; GitHub est le site qui héberge les dépôts en ligne et permet de les partager.
JSON	Format texte pour stocker des données sous forme d'objets (clé : valeur) et de listes. Ici, `operations.json` joue le rôle de base de données.
Module	Fichier JavaScript qui regroupe des fonctions autour d'une même responsabilité (stockage, registre, recherche, synthèse).
Mutation	Modification directe d'un tableau ou d'un objet existant (`push`, `sort`, `splice`), par opposition à la création d'une copie.
Node.js	Environnement qui permet d'exécuter du JavaScript en dehors d'un navigateur, ici dans le terminal.
Opération culturale	Intervention réalisée sur une culture : semis, arrosage, désherbage, etc.
Parcelle	Portion de terrain de l'exploitation (par exemple P1) sur laquelle on suit les opérations.
Push	Envoi des commits locaux vers le dépôt distant sur GitHub.
Terminal	Interface texte dans laquelle on tape des commandes ; l'application s'utilise ainsi, sans interface graphique.
Test unitaire	Petit programme qui vérifie automatiquement qu'une fonction donne le résultat attendu. Un test « au vert » est un test réussi.
`try` / `catch`	Le code essaie d'exécuter le bloc `try` ; en cas d'erreur, il exécute le bloc `catch` au lieu de planter.